

第一课：Agent 生态认知革命——从“对话”到“执行”的范式转移

01

课程主题

项目	内容
课程主题	Agent 生态认知革命
课时安排	第1周第1次课（2小时）
学员基础	熟悉Python编程，了解大模型基本原理
教学目标	1. Agent 引领技术演进新范式 2. 深度拆解 Agent 运作原理 3. 剖析 Agent 生命周期演进 4. OpenClaw 搭建与首次体验

02

环境准备

- Python 3.11+ 环境
- 大模型 API 密钥（OpenAI / 国产模型）
- OpenClaw 安装包与环境依赖

03

课程详情

第一部分：课程地图（10分钟）

1.1 开场白（5分钟）

【开场：引发思考】（0:00-0:45）



公元 1081 年春天，
眼瞅着快清明，
黄州东门外的水田里，
一位农夫正弯着腰吭哧吭哧插秧，
汗珠不停从它额头滴落。

他直起腰，抹了一把汗。

此时微风拂过，
他的思绪飘回到前些年曾经写给朋友的信。
他在信里说，
我发现土地丰收的秘诀，
不在于，把土地榨干，
也不在于，争抢农时，
而在于，等庄稼熟透了再收。
反之，如果你寸寸取之，日夜而又忘之，
种的时候，不赶农时，
收获的时候，又不等它成熟，
这还哪有什么好收成啊！

这个农民叫做苏东坡，
彼时他正在黄州，正经历他人生的至暗时刻。

尊重时间，耐心耕耘，
是我们这片土地最朴素的生存智慧。

我们是农耕文明，
我们对时间的理解，是从土地开始的。
春天播下种子，秋天收获粮食。
在这片土地上，
很难存在快捷方式，
也很少有速成的办法，
事情也不是你想做立马就能做成的。

AI 时代，我们每天都被各种速成信息蛊惑，
也被各种未经证实的暴富途径搞的心神不定，
注意力也很容易被娱乐和享受主义所吞噬。

但我认为大多成功的秘密，

藏在长期主义种下的那颗种子里。

山水迢迢之境，充满无限挑战。

做三、四月的事，八、九月自然就有答案。

在这里希望大家好好学习，

耐住寂寞，

积极乐观奋斗，

静待最终花开。

《翻页》

大家好，欢迎来到 AI Agent 全栈开发课程。我是这门课程的主讲老师——曾林。

在正式开始之前，我想请大家先静下来想一个问题：

过去这一年——也就是2025年——你让AI做过的最复杂、最超出预期的事是什么？

嗯～我猜一下：

- 也许是：你让它帮你写了一段复杂代码
- 也许是：你让它生成了一份专业报告
- 也可能是：你让它帮你整理了几百页的会议纪要
- 甚至——有没有同学尝试过让 AI 帮你完成一个完整的工作流？

大家不用着急回答我。

我们盘点下去年 2025年AI在普通人手中的真实使用场景，大家也可以想想并对比下今年，看看是不是 AI 进化速度已日新月异。

《翻页》

【虚拟互动：总结学员典型场景】（0:45-2:30）

概括来说，去年AI使用场景大致可以归为以下这几类：

第一类场景：生产力提效工具

比如：总结周报、做PPT、润色邮件、生成会议纪要等等这些日常高频场景。

2025年，很多人已经习惯了打开办公软件时，旁边有个AI助手随时待命。比如，飞书和钉钉内置的AI，可以直接从会议录音中提取待办事项，甚至自动生成会议纪要框架，我们只需要进行提示词微调即可。

第二类场景：学习研究助手

比如：让AI帮我们读论文、读财报、读长文档。这已经成了各行各业业务领域专家的标配。

使用 Kimi、豆包等产品的长文本处理能力，可以帮我们在几分钟之内深度剖析几十页 PDF。

但该阶段在大多数时候，我们还只是让 AI 帮我们梳理和总结，而真正写综述、做PPT、形成观点，大部分工作还得靠自己动手。

第三类场景：创意辅助。

比如：用 AI 生图、作曲、写歌、生成短视频、做游戏等等。

自从去年开始，创意类工作很大程度上已经离不开AI，但那时AI生成的还只是‘素材’，最终剪辑、修改、落地，仍然要靠我们自己。

第四类场景：生活管家

比如：用 AI 查天气、订机票、规划旅行路线，甚至用它辅导孩子作业。去年这些场景最初大多是单次问答，很少有多轮协作完成任务闭环。

讲到这里，大家发现没有？2025年这些 AI 场景都有一个共性：AI负责‘生成’，人类负责‘执行’。AI 只是给了一堆零件，组装大部分还得靠我们人类自身。

《翻页》

【深度：2025年 AI 应用全景扫描——从“对话”迈向“执行”】（2:30-4:15）

但是，除了上面这些场景之外，我们也惊喜发现一些‘另类’的AI应用在悄悄涌现——它们不再仅仅是‘生成’，而是开始‘执行’。

案例一：智能代码工厂——世界正在开始使用 AI 来重构软件研发

2025年，Anthropic Claude Code 与 OpenAI Codex 在超过200万家企业落地，覆盖从初创公司到财富 500 强的全规模软件团队。AI 编程已不再是简单的自动补全，而是能基于自然语言需求直接生成完整代码库、实时调试跨文件依赖、甚至根据 CI/CD 失败日志自主迭代修复。硅谷某 SaaS 独角兽披露，其工程团队在使用 Claude Code 后，新功能上线周期从6周缩短至4天，且 AI 生成的单元测试覆盖率达到 92%。更有开发者社区数据显示，Codex 在复杂架构重构任务中的代码采纳率已突破 65%。

所有这背后的核心变化就是：AI 已经不再单纯只是‘辅助编码’，而是真正做到能够‘主导工程’了。

案例二：数字人直播带货——让AI当主播

在2025年双十一，京东大模型已经应用到 1800 多个场景，服务超 300 万商家。数字人直播已经不再是简单的播报，而是能实时互动、智能调整话术、甚至根据观众反应调整促销策略。有商家反馈，数字人直播的效果超越了80%的真人主播。这背后的核心变化是：AI 也已经不再仅仅只是‘说话’，而是开始‘控场’。

案例三：医疗诊断的‘AI分身’——让AI代替‘跑腿’

在浙江的一些基层医院，AI医生分身已经可以独立帮病人完成初步问诊、解读病人体检报告、生成健康管理方案等等。比如：当老人测量血压后，AI 可以在1分钟完成自动分析数据、判断异常、并给出用药提醒——全程都不需要医生介入。这已经不再是简单的问答，而是完整的‘感知-决策-反馈’闭环。

案例四：机器人手机——让AI在移动端主动服务

去年的豆包手机，到今年3月荣耀在世界移动通信大会上展示了概念机——一款‘机器人手机’。它可以自主规划行程、自动叫车、甚至帮你排队买咖啡。媒体报道用了一个关键词：AI 已经正从‘云端对话’加速走向‘端侧执行’了。

【核心洞察：从“对话”到“执行”——AI智能体元年】（4:15-5:00）

以上这些案例，虽然领域各个不同，但却都指向同一个趋势。

那就是 AI 正一步步加速进化和演进，从最初只会‘能说会道’正快步走向‘自主办事’。

传统模式下，我们和 AI 的交互方式是‘你问我答，你指挥我执行’。AI 和人类五五协同，比如：AI 提供信息和建议，我们人类动手执行。我们和AI的关系还是朋友关系。

但在这些新场景里，AI开始承担执行者的角色：它自己控制代码生成，数字人直播，健康管理，甚至自己买咖啡。

这种转变意味着什么？

——意味着2026年，我们已经开始真正进入‘AI智能体元年’。

但这也产生了一个细思极恐的事情，那就是AI开始正在慢慢加速跟人类抢夺生存空间。

很难讲，这究竟到底是好，还是坏。但无论好与坏，趋势开始不可逆。

从这里我们来看，智能体（Agent）和传统AI最大的区别，就是它开始具备自主性思维，可以主动规划、调用工具、执行任务、闭环反馈。它已经不再仅仅只是一个被动的‘对话机器人’，而变成了一个有一定思想的主动的‘执行者’。

《翻页》

1.2 课程地图（5分钟）

【过渡：从“为什么学”到“学什么”】（0:00-0:30）

好，上面我们已经聊了今年2026年是AI智能体元年，也看到了AI已经从‘对话’到‘执行’的质变。那么紧接着，问题就来了：

1. 这样一门课，我们到底要学什么？
2. 今天第一堂课，又会带大家走到哪里？

不卖关子，我们先给出一张课程地图，让大家先对今天这节课有一个宏观感知——就像玩游戏要先看小地图，知道自己在哪、要去哪。

学习也是一样。

本节课，地图一共有五个站点。

《翻页》

【第一站：认知重构——Agent 的诞生与爆发】（0:30-1:15）

第一站：认知重构——Agent 的诞生与爆发

这个站点我们会抛出一个观念：那就是AI不应该只是一个‘问答盒子’，而应该是一个能主动行动的实体。

在这一趴，我会带大家看清三件事：

1. 第一件事：从 ‘Copilot’ 到 ‘Autopilot’ 的范式转移到底是什么？
2. 第二件事：为什么从2025年下半年开始，所有大厂都在 All in Agent？
3. 第三件事：Agent 爆发的三大技术必然性——注意，这不是玄学，而是实打实的技术拐点。

相信大家听完第一小节，就会对 Agent 有一个高屋建瓴的认知，不会再把它当成一个简单的 ‘自动化脚本’ 。

【第二站：深度拆解——Agent 如何运作？】（1:15-2:15）

第二站：深度拆解——Agent 如何运作？

这一站是理论硬核时间，我们会把 Agent ‘开膛破肚’，看看它肚子里到底装了些什么东西。

这里咱们先给出一个核心公式，那就是：

代码块

```
1 Agent = 大脑 (LLM) + 规划 (Planning) + 记忆 (Memory) + 工具调用 (Tool Use)
```

在这一小节，我们将逐一拆解：

- 第一：大模型到底怎么理解模糊指令？
- 第二：规划层是怎么把一个 ‘帮我安排出差’ 拆成多个子任务的？
- 第三：记忆层到底是怎么记住你上次的偏好的？
- 第四：工具层到底怎么调用API、以及执行代码、甚至直接操作你的电脑？

更重要的是，我们会深入 ReAct 模式，该模式是思考-行动-观察的循环，也是 Agent 能持续工作的 ‘心脏起搏器’。再配合自我反思机制，就可以让智能体 Agent 学会自己纠错。

这一节听起来会有点抽象，但我尽量会用最直观的案例让你听懂。

听完之后，当你再看任何 Agent 产品，就能一眼看穿它的底层架构。

【第三站：技术演进——从API到Agent的必然之路】（2:15-3:00）

地图的第三站是技术演进——从API到Agent的必然之路

有了前面两站的 Agent 基本认知和了解它底层的机制原理后，我们紧接着梳理一条 Agent 技术演进时间线：那就是为什么2022年 Agent 没爆发，2023、2024年也没有，而为什么偏偏是 2025-2026 年开

始疯狂爆发？

在这期间，我会带大家走过四个阶段：

- 第一个阶段：2022-2023 年基础API调用阶段，这一阶段是人类手动编排阶段
- 第二个阶段：2023-2024 年提示词工程阶段，在这个阶段我们让模型学会听话
- 第三个阶段：2024-2025 年 Function Calling 阶段，在这个阶段我们模型学会 ‘动手’
- 第四个阶段：2025-2026 年 Agent框架阶段，在这个阶段我们让模型不仅学会动手，也学会了自主思考

这里，关键转折点是 Function Calling 的诞生。

它让模型从输出文字，变成输出结构化的函数调用指令，这是 Agent 自己 ‘动手’ 的最早期技术萌芽。

在这期间，也诞生了两个工程化协议：MCP（模型上下文协议）和 A2A（智能体间通信协议）。

它们就像 Agent 世界的 HTTP 和 TCP/IP，AI 的工程化正在孵化和构建下一代智能体协同网络。

相信大家听完这一小节，不仅会知道 Agent 是怎么来的，还能预判它未来往哪里去。

【第四站：Agent 实践启航 —— OpenClaw 环境搭建与初体验】（3:00-4:30）

紧接着，我们来到第四站。Agent 实践启航 —— OpenClaw 环境搭建与初体验

前面讲了这么多理论，终于要动手了！这一节我们会一起完成三件事：

第一：认识 OpenClaw——这个GitHub上28.5万星标的 ‘龙虾’ 框架，到底凭什么这么火？它的核心设计哲学是什么？

第二：环境搭建——我会手把手带大家克隆代码、创建虚拟环境、配置API密钥。这个过程可能会遇到各种坑（网络问题、依赖冲突等等），但我会把常见问题的解决方案提前给到大家，争取让每个人都能顺利跑起来。

第三：我们会手搓第一个Agent——我们会写两个极简示例：

- 一个 ‘Hello World’ Agent，让大家感受最基本的对话能力
- 以及一个带工具的 Agent，让它学会调用外部函数，比如获取当前时间。

当大家看到 Agent 自己调用工具、并把结果整合进回答时，我相信，那种 ‘wow，它真的在思考’ 的感觉，就是 Agent 最大魅力的开始。

而且这里，我们还会做一个小对比：同样一个任务，用传统代码写要多少行，用 OpenClaw 写要多少行——你会发现，Agent 框架不仅是 ‘另一种写法’，而是 ‘另一种思维’。

【终点站：本节课程总结与下节预习任务】（4:30-5:00）

讲完这些，第一课基本就要告一段落了，我们会做课程总结以及下一节课的预习任务。

我们会快速回顾今天学到的核心知识点：Agent 的定义、四要素、ReAct 模式、技术演进四阶段。然后回顾下我们实现的两个小任务：

1. 第一个小任务是完成 OpenClaw 的环境搭建，确保能运行基础示例。
2. 第二个小任务是接入国产大模型，并自定义一个简单工具。该应用我们会基于 LangChain 框架构建完整 Agent，并深入理解 Memory 机制，并跟 OpenClaw 做对比。

好，本节课地图画完了，整堂课的结构就是这五步，环环相扣。

现在，相信大家心里应该有数了——我们要去的是一个什么样的地方，以及怎么走。

【结尾互动】（5:00）

那么，准备好了吗？我们现在就出发，进入第一站：认知重构——Agent 的诞生与爆发。

《翻页》

第二部分：认知重构——Agent 从诞生到出圈（20分钟）

我们先进入第一站：认知重构——Agent 从诞生到出圈

《翻页》

2.1 从"Copilot"到"Autopilot"的范式转移（10分钟）

【开场：承接上一节，引入主题】（0:00-0:45）

第一站我们首先要明确 Agent：它不再像大模型，仅仅只是一个被动的‘问答盒子’，而是它正在经历一场深刻的角色转变。为了让大家更直观地理解这个转变，我们先来看两个场景的对比。

《翻页》

【案例对比：传统 AI vs AI Agent】（0:45-3:00）

我们来看这一页 PPT，我们先看左边：

“假设现在是2023年，你打开 ChatGPT，输入：

帮我查一下北京的天气

AI的回复很标准：

AI：杭州今天晴，气温5-15°C，风力2级，适合户外活动。

——结束。接下来你要根据这个信息自己做决定：穿什么衣服？要不要带伞？需不需要调整行程？

这是典型的传统 AI 大模型信息交互模式：AI提供信息，人类则负责思考和执行。

接着我们再看右边：

现在来到2026年，你面对的是一个Agent智能体。你输入：

帮我策划明天的出差行程

你会看到 Agent 这样的回复：

1. 智能体首先调用天气API获取目的地天气（比如：确认杭州明天晴，温度 5-15°C）
2. 智能体第二步根据天气推荐着装建议（比如：薄外套+衬衫，无需雨具）
3. 接着智能体第三步打开携程查询高铁/航班（比如：推荐G5次高铁，7:00出发，11:30到达，商务座有余票）
4. 智能体第四步在日历中创建行程提醒（比如：已创建‘杭州出差’日程，提醒时间明早6:00）
5. 然后来到第五步，智能体预约接送机服务（比如：已预约专车，明早5:30到达您家）

完成！智能体最后发现所有安排都已就绪，并且把整个规划都同步给到我们？

——看到了吗？Agent 不再是给你一堆信息，然后让你自己将这些琐碎信息串联、决策和动手，而是非常贴心地直接帮你把所有事情都事无巨细地办妥了，就像你的私人秘书一样。

《翻页》

【深入剖析：两个案例的本质差异】（3:00-6:00）

好，我们来拆解一下，看看传统 AI 大模型案例和现代智能体 Agent 的本质差异到底体现在哪些地方：

第一个差异：AI 角色变了。

在传统模式下，AI是副驾驶（Copilot）——它坐在副驾驶位置上，给你看地图、报路况，但方向盘还在你手里。它说‘前面该右转了’，你根据它给你的信息做抉择和行动，你得自己打方向盘。

但是在 Agent 模式下，AI 是自动驾驶（Autopilot）——你告诉它目的地，它自己规划路线、控制油门刹车、避开拥堵，全程不需要你干预，你完全可以放任托管，甚至睡觉。此时它不再是建议者，而是切实的执行者。

第二个差异：交互的深度变了。

传统交互是单次往返：你问一句，它答一句。交互流是断开的。

而 Agent 交互则是持续闭环：它把一个大目标拆成多个子任务，并且按顺序进行执行，每一步结果都会影响下一步的决策。它记得天气的信息，所以才能推荐着装；它知道出发时间，才能预约接送机。这是一个完整的【思考-行动-观察】循环。

第三个差异：价值产出变了。

传统AI产出的是信息——天气数据、航班列表。这些信息还需要你二次加工才能变成行动。

相比而言，Agent 产出的是结果——比如：行程已安排好、日历已创建、车已约好。你省去了从信息到执行的转换成本。

这里用一个形象的比喻来比较那就是：

传统 AI 更像是一个顾问——你提出问题，它帮你谋划，然后给出信息结果，你针对结果做决策然后自己执行。

而 Agent 则像是一个能干的管家——比如：你说‘我要招待客人’，它自己列菜单、买菜、做饭、摆盘，最后端上桌。

《翻页》

【虚拟互动】（6:00-7:00）

这里也许有喜欢思考的同学会问了：‘Agent 是怎么做到的呢？它到底凭什么能调用这么多不同的服务呢？’

——这是个好问题！这也正是 Agent 核心能力所在。

Agent 之所以能‘动手’，是因为它具备以下三个关键能力：

1. **规划能力：比如：**把‘策划出差行程’这个大目标，自动拆解成‘查天气’、‘查交通’、‘创建日历’、‘约车’这几个子任务。而且这不是程序写死的流程，而是大模型实时动态生成的计划。

2. **工具调用能力：智能体**它知道什么任务该用什么工具——比如：查天气就用天气API，查航班就用携程API，比如创建日历就用日历API，约车则用打车API。这里每个 API 就像一个‘技能插件’，Agent 针对这些插件可以动态调用。
3. **记忆能力：智能体**它记得你查的天气是杭州，所以推荐着装时不会推荐羽绒服；它记得你选了高铁G5次，所以约车时约的是5:30出发去火车站。这种跨步骤的记忆，可以让整个执行过程准确，而且连贯流畅。

这三个能力加起来，就是 Agent 能从一个无脑‘对话机器人’进化成一个懂你的‘执行管家’的技术底座。

《翻页》

【范式转移的本质：从被动到主动，从建议到执行】（7:00-8:30）

所以，从 Copilot（副驾）到 Autopilot（自驾）的范式转移，本质上是 AI 角色的四重根本性转变：

- 第一重转变：从被动响应到主动规划：传统AI等着你提问，Agent 自己则生成行动步骤。
- 第二重转变：从信息提供到任务执行：传统AI返回给你信息，Agent直接帮你把事情办了。
- 第三重转变：从单次交互到持续闭环：传统AI每次对话都是一次交互，而Agent能记住上下文，持续跟进直到所有任务完成。
- 第四重转变：从辅助工具到执行主体：传统AI是工具，人类才是主体；而 Agent 出现后，则开始成为执行主体，人类只需要下达指令。

这种转变，意味着AI的定位正在从‘提升效率的工具’变成‘替代执行的主体’。它不是帮你更快地做事，而是直接帮你做事。

我还记得，2025年有一句行业名言，那就是‘未来的AI，不再是仅仅帮你写周报了，而是帮你完成周报里所有的工作。’这句话听起来，老实说，挺让人脊背发凉，如果真的有这一天到来，那么人类将丧失很多生存的尊严。

但这个趋势已经像射出去的箭，无法再掉头扭转。也许潘多拉魔盒已经开启了也未可知……

《翻页》

2.2 2026年Agent爆发的三大技术必然性（10分钟）

【开场：承上启下】（0:00-1:00）

好，上面我们已经理解了从 Copilot（副驾）到 Autopilot（智驾）的范式转移——AI正在从‘信息交互’走向‘自动化执行’。那么问题来了：为什么这个转变恰好发生在2026年？为什么不是2023年，也不是2025年？

下面，我们就来拆解其背后的三条历史必然。这不是讲故事，而是技术演进的自然结果。只有理解了它们，大家就能看懂整个行业的发展脉络。

【第一条必然性：模型上下文能力突破百万级 token】（1:00-4:00）

先解释下什么是上下文？简单来说，就是模型一次能记住多少信息。就像人的工作记忆，如果只能记住几句话，就很难处理复杂的任务。

在 2025 年上半年，主流大模型的上下文窗口还停留在20万Tokens左右——20万 tokens 是个什么概念，大概也就是《三体》这本畅销书三部曲的体量。听起来似乎不少，但是对于真正的复杂任务，20万Token根本兜不住。让我们举个例子：

假设你想让AI帮你分析一家上市公司的十年财报，并写一份投资建议报告。十年的年报，光是文字部分就可能超过50万Tokens。如果用20万Token的模型，AI读到后面就会忘记前面——就像一个人只能记住前三章的内容，后七章都成了‘断片’。

但时间来到 2025 年下半年，情况就发生了质变。OpenAI、Google、Anthropic以及国内的智谱、月之暗面，纷纷将上下文扩展到 100 万到 200万Tokens。这意味着什么？

- 你可以把《二十四史》一次性喂给AI，它仍能记住第一部开头的细节
- 你可以把长达一小时的会议录音转录文本全部丢进去，AI可以基于全程信息生成会议纪要
- 最重要的是，这时的 Agent 在执行复杂任务时，可以在半小时到一小时内持续跟踪状态——包括：每一步的思考、每一个中间结果、每一次工具调用，它都不会丢失细节信息，它可以持续执行而不再‘断片’

要知道，在没有长上下文之前，Agent 经常面临一个尴尬，那就是执行到第5步，却忘了第1步的结果。比如，一个市场调研 Agent 需要先搜索行业报告、再分析竞争对手、最后生成PPT。如果没有足够长的上下文，它可能在分析竞争对手时忘了行业报告的细节，结果就会导致结论前后矛盾。

现在呢，百万级上下文 token 可以让 Agent 能够‘包含’整个任务状态持续执行。它可以随时回顾之前的推理，也可以把中间结果存储下来供后续使用。这就像一个人拥有了过目不忘的能力，再复杂的任务也能有条不紊地推进。

所以，第一个必然结果就很明确了：上下文能力的突破，为 Agent 的长时间、多步骤执行扫清了记忆障碍。这也是底层能力的质变。

《翻页》

【第二条必然性：从 LLM 到 LAM 的架构演进】（4:00-7:00）

LLM 我们很熟悉——大语言模型的简称，它擅长生成文本、回答问题。但要知道智能体 Agent 需要的不光是‘会说’，还要求‘会做’。这就催生了新一代模型架构：LAM——大操作模型（Large Action Model）。

传统的 LLM，输入是文本，输出也是文本。你可以问它‘北京天气怎么样’，它给你一段描述。至于你接下来要不要带伞、要不要取消户外活动，模型不关心，也管不着。

但 LAM 不一样。它的输出不再局限于文本，而是可执行的操作序列。当你问‘帮我策划出差行程’时，LAM 内部生成的可能是这样一串指令：

代码块

```
1  [
2      {"action": "call_api", "tool": "weather", "params": {"city": "北京"}},
3      {"action": "call_api", "tool": "flight_search", "params": {"from": "上海",
4          "to": "北京", "date": "明天"}},
5      {"action": "create_calendar_event", "params": {"title": "北京出差", "time":
6          "明天 7:00"}},
7      {"action": "book_car", "params": {"pickup_time": "明天 5:30"}}
8  ]
```

这些指令可以直接交给执行引擎，调用各种 API、操作各种应用。此时 LAM 不再是‘建议者’，而是‘指挥官’。

为什么会有这种演进？因为 AI 主流大模型供应商开始意识到，仅仅生成语言是不够的。真正的智能必须能改变世界，而改变世界需要行动，而不只是瞎 BB。

于是，模型训练的目标开始从‘预测下一个词’逐渐扩展到‘预测下一步操作’。体现在：

- 第一：OpenAI 的 o1 系列引入了‘思维链’推理，让模型能一步步思考
- 第二：Anthropic 的 Claude 3.5 Sonnet 强化了工具调用能力
- 第三：国内 Qwen、DeepSeek 等模型也纷纷在训练中加入了大量‘操作类’数据，教模型如何调用工具、如何规划步骤等等。

时间来到 2026 年，LAM 已经成为新一代模型的标准配置。Agent 不再是‘LLM + 工具调用’的拼凑，而是天然具备操作能力的一体化架构。这就像早期的汽车需要司机手动摇把启动，而现在的汽车一键启

动——因为底层架构已经为自动化做好了准备。

所以，Agent 范式转移的第二个必然性就是：模型架构从‘对话’转向‘操作’，Agent 有了‘动手’的先天能力。

《翻页》

【第三条必然性：开源生态的引爆效应】（7:00-9:30）

前面两条讲的是模型本身的能力，但能力再强，也需要一个落地的平台。2025年11月，一个开源框架横空出世，彻底引爆了 Agent 生态——它就是大名鼎鼎的 OpenClaw，也就是大家熟知的龙虾。

截至2026年3月，OpenClaw在GitHub上的星标数达到28.5万。这个数字是什么概念？我们可以对比一下：

- Linux内核，开源史上最成功的项目之一，星标数约10万；
- VS Code，开发者最爱的编辑器，星标数约15万；
- 而OpenClaw，仅用4个月就冲到28.5万，创下了开源软件历史最高纪录。

为什么它能这么火？因为它彻底解决了开发者和使用者最痛的几个问题：

1. **第一个问题：开箱即用：**OpenClaw 内置了规划、记忆、工具调用、多 Agent 协作等核心能力，开发者只需要配置 API 密钥，就能跑起一个能干的 Agent。这是多么方便的使用体验。
2. **第二个问题：多模型支持：**它可以同时接入GPT、Claude、Gemini、国产模型，甚至可以在不同任务中动态路由。
3. **第三个问题：生态丰富：**OpenClaw 引入了‘技能商店’（ClawHub），该商店提供了上千个预置工具，从查天气到操作 Excel，一键安装就能用。

在2026年1月的CES展，也就是国际消费电子展，戏称科技春晚上，英伟达创始人黄仁勋在被问及 Agent 未来时，说了一句发人深省的话，被媒体疯狂转载。他说：

‘OpenClaw是迄今发布过的最重要软件，它让每个人都能拥有自己的智能体，就像当年互联网让每个人都能拥有自己的网站一样。

这话虽然听起来有点夸张，因为我们都知道黄教主销售能力一流，不折不扣的商业变色龙。但仔细想想，他说的也确实有几分道理。

互联网时代，Apache、Nginx等开源软件让建站变得轻而易举，催生了整个Web生态。而OpenClaw正在扮演同样的角色——它把Agent的开发门槛从‘需要顶尖团队’降到‘一个程序员一下午搞定’。

OpenClaw 开源意味着什么？意味着全球的开发者都可以参与贡献、分享技能、修复bug。OpenClaw 发布后，短短几个月内：

- 社区贡献了3000多个自定义Skill
- 涌现出大量基于OpenClaw的垂直Agent应用（数据分析Agent、销售助手Agent、教育辅导Agent等等）
- 甚至出现了专门为OpenClaw做模型微调的创业公司

这种自下而上的生态力量，是任何一家公司靠封闭研发都无法复制的。当数十万开发者同时在为Agent生态添砖加瓦时，技术迭代的速度将是指数级的。

所以，2026年智能体爆发的第三条必然性是：OpenClaw 等开源框架爆发，让Agent从实验室走向了大众，形成了滚雪球式的生态效应。

《翻页》

【小结与过渡】（9:30-10:00）

好了，我们再来简单回顾一下2026年Agent爆发的三大技术必然性：

- 第一，上下文能力突破百万级——让Agent能够长时间持续执行，不再‘断片’。
- 第二，从LLM到LAM的架构演进——让Agent天然具备操作能力，而不只是‘说话’。
- 第三，开源生态的引爆效应——OpenClaw等框架大幅降低了开发门槛，形成了全球协作的创新网络。

这三条，一条是‘记忆力’的突破，一条是‘动手能力’的突破，一条是‘生态土壤’的爆发。这三条是宏观层面的驱动力。三者叠加，让2026年最终成为Agent落地并开始爆发的元年。

《翻页》

第三部分：深度拆解——Agent如何运作？（30分钟）

3.1 Agent的底层公式（10分钟）

【开场：承接上节，引入主题】（0:00-1:00）

了解了‘为什么 2026年是Agent爆发之年’，接下来我们就要深入内部，进入到微观层面，看看Agent到底是怎么工作的。我们将打开Agent的黑盒子，看看它的‘大脑’、‘四肢’和‘记忆’是如何进行协同的。

以下就是 Agent 公式，也称为 Agent 四要素：

代码块

```
1 Agent = LLM (大脑) + Planning (规划) + Memory (记忆) + Tool Use (工具调用)
```

这个公式不是我发明的，而是业界公认的Agent最小构成单元。任何一个真正的智能体，都必须具备这四个组件。缺一个，它就不是完整的Agent，最多算一个增强版的聊天机器人。

接下来，我们逐一解剖这四个组件，看看它们各自扮演什么角色。

《翻页》

【组件一：LLM——Agent的大脑】（1:00-2:30）

第一个组件：LLM，全称大语言模型。这是Agent的‘大脑’。

就像人的大脑负责思考、决策和理解语言，LLM在Agent中承担着最核心的认知功能，其中包括：

- 第一：意图理解：当你对Agent说‘帮我策划出差行程’，LLM首先要理解这句话的真正意图——不是闲聊，不是查天气，而是要完成一整套任务。
- 第二：推理与规划：大语言模型要根据目标生成行动计划，决定先做什么、后做什么。
- 第三：生成与输出：无论是调用工具的指令，还是最终回复用户的语言，LLM都要生成并输出内容。

举个例子：假设你对Agent说‘我下周要去北京出差，帮我准备一下’。LLM会立刻在内部进行一系列推理：

- ‘下周’具体是哪天？需要确认。
- ‘准备’包括哪些？机票、酒店、行程、天气……
- 我需要调用什么工具来获取这些信息？

这个推理过程，就像你脑子里在默默列清单。没有LLM，Agent就是个聋子瞎子，既听不懂指令，也说不出结果。

但注意：LLM再聪明，也只是一个大脑。一个人如果只有大脑，没有手脚，没有记忆，他什么都做不了。这就引出了其他三个组件。

《翻页》

【组件二：Planning——Agent的执行导演】（2:30-4:30）

第二个组件：Planning，规划能力。这是Agent的‘执行导演’。

为什么需要规划？因为真实世界的任务往往是模糊的、复杂的，需要拆解成一系列可执行的子任务。

假设你让一个没有规划能力的Agent‘帮我安排下周的部门团建’。它可能会直接调用一个‘团建安排’工具——但问题是没有这样的工具。或者它干脆说‘我做不到’。

但一个有规划能力的Agent会怎么做？

它会自动拆解：

1. 第一步：确定时间。它会询问大家的时间，或根据日历找出空闲时段。
2. 第二步：收集需求。大家想玩什么？吃饭还是户外？
3. 第三步：搜索场地。根据预算和人数查找合适的团建地点。
4. 第四步：预订。打电话或在线预订。
5. 第五步：通知。它会生成通知发送给相关人。

这个过程并不是写死的代码，而是LLM根据任务实时生成的动态计划。每一步执行完后，Agent还会根据结果调整后续计划——比如发现预算超了，就换一个便宜点的场地等等。

这种动态规划能力，正是Agent区别于传统自动化脚本的核心。传统脚本是‘if-this-then-that’（也就是传统条件语句穷举所有可能性），而Agent则是‘let-me-think-how-to-do-it’（也就是自主思考）。

规划的模式有很多种，比如 ReAct（思考-行动-观察）、Plan-and-Execute（先计划后执行）、Self-Ask（自问自答）。我们会在后面的课程里专门讲这些模式。今天你只需要记住：规划是让Agent能够处理复杂任务的‘大脑皮层’。

《翻页》

【组件三：Memory——Agent的记忆模块】（4:30-6:30）

第三个组件：Memory，记忆。这是Agent的‘海马体’。

记忆分为两种：短期记忆和长期记忆。

短期记忆就是对话上下文。比如你告诉Agent‘我叫张三，喜欢靠窗的座位’。在整个对话过程中，Agent会记住这些信息，下次订机票时会自动选靠窗。但对话只要一结束，短期记忆就清空了。

长期记忆则存储在外部数据库（比如向量数据库）中。Agent可以把你的偏好（‘张三，靠窗，素食主义者’）保存下来，下次再找你对话时，它会主动加载这些信息，加载信息后的agent就像你的老朋友一样，对你了如指掌。

为什么记忆如此重要呢？

想象一下，如果Agent没有记忆，那么每次对话都是‘陌生人’——你每次都要重新告诉它你的名字、你的喜好、你之前聊过什么。这就像每次去医院都要重新填病历，效率极低。

更重要的是，在执行复杂任务时，记忆能保证连续性。比如一个调研Agent，它先搜索了行业报告，然后分析竞争对手，最后生成PPT。如果没有记忆，它可能在做PPT时忘了行业报告的结论。有了记忆，它可以随时回顾之前的结果，确保前后一致。

2026年的Agent，记忆能力已经非常成熟了。短期记忆由大模型的上下文百万级窗口支撑，长期记忆则由向量数据库（如Pinecone、Chroma）或内置存储实现。有了记忆，一些Agent甚至能记住你几个月前的对话，并在适当时机主动提起——这才是真正的‘智能助理’。

《翻页》

【组件四：Tool ——Agent的手脚】（6:30-8:00）

最后一个组件，也就是第四个组件：Tools，工具调用。这是Agent的‘手脚’。

大脑再聪明，规划再周密，记忆再清晰，如果没有手脚，Agent永远停留在‘纸上谈兵’——它只能给你建议，无法真正改变世界。

工具调用可以让Agent能够：

- 第一：调用API：比如：查天气、订机票、发邮件等等
- 第二：执行代码：比如运行Python脚本、处理数据等等
- 第三：操作应用：比如可以帮你打开Excel、在日历上创建事件、控制智能家居等等
- 第四：访问网络：比如搜索信息、抓取网页

工具调用的技术基石是 Function Calling。2024年OpenAI推出Function Calling后，所有主流模型都支持了类似功能。它的本质是让模型输出结构化的函数调用指令，而不是纯文本。

比如，当大模型在获取了用户的查天气这个需求后，可能会输出以下格式化指令字符串：

代码块

```
1  {
2      "function": "get_weather",
3      "parameters": {
4          "city": "北京",
5          "date": "2026-03-18"
6      }
7  }
```

执行引擎接收到这个指令，就去调用真实的天气API，然后把结果返回给LLM，LLM决定是否再继续下一步。

工具的可扩展性是Agent生态繁荣的关键。龙虾OpenClaw的工具仓库已经有上千个预置工具，覆盖了从开发到生活的方方面面。当然你也可以自己写一个工具，让Agent学会做任何你想自动化的事情，

这在我们以后课程也会专门介绍。

《翻页》

【深度洞察：四者缺一不可】（8:00-9:30）

好，四个组件都讲完了。现在我们来做一个思想实验：如果缺少其中某一个，会发生什么？

- 如果只有LLM，没有Planning，规划：Agent遇到复杂任务就会‘懵’。你让它‘策划团建’，它可能直接回答‘我没法做，需要你告诉我具体步骤’。或者它尝试调用一个不存在的一步到位工具，然后失败。没有规划，Agent只能处理那些可以直接映射到单个工具的任务，无法应对多步骤的复杂场景。
- 如果只有LLM和Planning，没有Memory：Agent会在执行过程中‘失忆’。它规划得很好，但走到第三步就忘了第一步的结果。比如订机票时，它查了你的偏好（靠窗），但到下单时却忘了，给你订了个过道。每次对话都像是‘刚认识’，无法积累历史过往，个性化服务也就成为了空谈。
- 如果只有LLM、Planning、Memory，没有Tool 调用：Agent就成了一个‘空想家’。它能理解你的需求，能制定完美计划，能记住所有细节，但就是没法执行——因为它没有手脚。它只能给你一份详细的《如何安排团建》报告，然后你自己去一个个操作。这仍然是信息交互，不是自动化执行。

所以，这四个组件就像一辆车的四个轮子，缺一个都没法跑起来。LLM提供智能，规划提供策略，记忆提供连续性，工具提供行动力。只有四者结合，AI才能从‘对话者’进化为真正的‘执行者’。

《翻页》

【小结与过渡】（9:30-10:00）

上面我们拆解了Agent的底层公式，我们知道了：

- LLM，就是大脑，它负责理解、推理、生成
- Planning，就是执行导演——它负责任务分解、动态调整
- Memory，类似海马体——负责短期和长期记忆
- Tool，工具集——它负责调用API、执行代码、操作应用

这四个组件共同构成了一个完整的智能体。理解了它们，你就掌握了Agent的基本框架。

那么，这个框架在实际中是如何运作的？Agent的‘思考’过程到底长什么样？下面，我们将深入ReAct模式——看看Agent是如何在‘**思考-行动-观察**’的循环中一步步完成任务的。

下面我们进行深度拆解——看看 Agent 是如何运作的？

3.2 Agent任务解构：从模糊指令到精确执行（15分钟）

3.2.1 Agent 任务解构：定义-感知-控制三维驱动框架

【开场：从“模糊指令”到“精确执行”】（0:00-1:00）

想象一下，你刚认识一个助理，你对他说：‘帮我分析一下最近的股票市场。’——这个指令有多模糊？哪个市场？什么时间范围？分析什么维度？用表格还是报告？助理如果直接动手，大概率会跑偏。

但一个成熟的Agent不会这样。它会先解构任务。Agent内部常见的任务解构框架——我称之为 ‘三维驱动框架’。该框架的示意图我展示在 PPT 上，大家可以仔细看一下。

代码块

- 1 任务解构框架
- 2 |—— **任务定义层**：明确目标与成功标准
- 3 |—— **状态感知层**：构建执行上下文（文件、日志、历史）
- 4 |—— **流程控制层**：规划执行路径与退出条件

由该图可知，Agent 的任务框架从逻辑上分为三层，分别是：

- 1. 第一：任务定义层，它负责明确目标与成功标准
- 2. 第二：状态感知层：它负责构建执行上下文
- 3. 第三：**流程控制层**：它负责规划执行路径与退出条件

这三层，就像建筑工程中的三张蓝图：首先我们需要先搞清楚要盖什么（对应定义层），接着再看地基和周边环境（对应状态感知层），最后执行设计和具体施工步骤（对应流程控制层），三者缺一不可。

接下来，我用一个真实案例按照 Agent 三层框架进行一一拆解。

这个案例就是：‘股票市场分析’ 案例。

【第一层：任务定义层——明确目标与成功标准】（1:00-3:30）

第一层：任务定义层。这是Agent接到指令后的第一反应：用户到底想要什么？

现实业务场景，客户原始指令往往是非常模糊的，比如用户会问：‘最近的股票市场怎么样’——‘最近’是多久？一周、一个月、还是年初至今？‘分析’是指罗列数据，还是给出投资建议？‘股票市场’是全球市场，还是A股、港股、美股？

如果Agent直接行动，很可能费力不讨好。所以，它需要澄清。

澄清的方式是什么呢，通过反问与引导。

一个设计良好的 Agent 会主动提问用户，目的是把模糊的目标转化为自己可理解、可执行的清晰指令。比如：

Agent反问用户：‘您想分析哪个市场？A股、港股还是美股？关注哪些板块或个股？需要我重点关注什么指标——涨跌幅、成交量、还是估值水平？分析结果希望以什么形式呈现——文字摘要、数据表格，还是可视化图表？等等’

这一连串提问，其实是智能体 Agent 在帮用户把‘模糊的需求’转化成‘具体的任务规格’。

除了澄清目标，任务定义层还要明确‘怎样算完成’。是生成一份报告就行，还是需要定时推送每日更新？是要推送到微信，还是保存在云端？

常见的情况是，在股票分析这个案例中，成功标准可能是：‘生成一份包含A股主要指数涨跌幅、板块轮动热力图、以及北向资金流向的分析报告，以PDF格式发送到我的邮箱，并在微信上通知我。’

只有清晰的目标和成功标准，智能体 Agent 才能确切知道自己该往哪走，以及什么时候可以停下来。

《翻页》

【第二层：状态感知层——构建执行上下文】（3:30-6:00）

任务定义层确定好后，下面我们进入 Agent 任务框架的第二层。

第二层是状态感知层。当任务目标明确后，接下来智能体 Agent 需要‘打量环境’——它需要感知当前的状态，并开始构建执行上下文。

状态感知包括两部分：

1. 第一：用户相关的上下文
2. 第二：环境相关的上下文。

用户上下文是指：跟用户有关的上下文。

比如智能体 Agent 会检索与用户相关的对话历史、用户的偏好、甚至之前保存的已执行的任务记录。比如：

- 这个用户是不是之前问过‘新能源板块’？是不是一直关注宁德时代、比亚迪？

- 这个用户的习惯是看日线数据还是看周线？喜欢用表格还是图表？
- 该用户是不是风险偏好型投资者？从历史对话中能不能推断出他的投资风格？

这些信息可能存储在长期记忆（向量数据库）中，智能体 Agent 执行任务时，会主动加载，从而避免重复提问。

环境上下文：是指跟环境相关的上下文。

比如智能体 Agent 需要感知当前的外部环境：

- 比如当前时间：如果是下午3点以后，A股已经收盘，那么‘最近’就应该包含今天的收盘数据；如果是上午10点，那么盘中实时数据可能更重要。
- 除了时间，环境上下文还包括市场状态：今天是不是节假日？有没有重大新闻（比如美联储议息、政策发布）？这些都会影响分析的角度等等。
- 此外，是否有第三方依赖资源，比如智能体 Agent 有没有访问实时股票API的权限？有没有连接金融数据服务（如Wind、彭博）的MCP协议？如果没有，它需要调整计划，改用免费数据源。

环境结果最终会注入给执行上下文，也就是说所有感知到的信息都会被整理成一个‘上下文对象’，传递给后续的流程控制层。比如：

代码块

```
1  上下文 = {
2      "用户偏好": {"关注板块": ["新能源", "AI芯片"], "输出格式": "图表+摘要"},
3      "当前时间": "2026-03-18 10:30",
4      "市场状态": "A股交易时段",
5      "可用工具": ["新浪财经API", "MCP金融协议", "matplotlib"]
6  }
```

这个上下文，就是Agent接下来做决策的依据。

《翻页》

【第三层：流程控制层——规划执行路径与退出条件】（6:00-9:00）

下面来到智能体 Agent 任务框架的第三层，也是最后一层：流程控制层。

当已经有了清晰的目标和丰富的上下文之后，Agent 终于可以开始规划‘怎么做’了。

这一层的输出是一个可执行的路径规划——有可能是一个步骤列表，也有可能是一个动态的决策树。Agent会按照规划一步步执行，并在每一步根据结果调整后续的步骤。

回到我们的上面的股票分析案例。假设经过前两层，Agent已经明确：用户要分析A股市场，需要关注新能源和AI芯片板块，最后输出图表和摘要。那么它会规划出类似这样的路径：

1. 调用股票API获取实时数据

- 比如使用新浪财经API或腾讯财经API，获取上证指数、深证成指、创业板指数的最新数据
- 接着获取新能源板块（如宁德时代、比亚迪）和AI芯片板块（如寒武纪、海光信息）的个股行情
- 调用 API 时，如果发现API有额度限制，可能调整降级为访问公开数据源。

2. 使用MCP协议连接金融数据服务

- Agent 直接连接专业金融数据库，获取机构级别的财务数据、研报摘要。

3. 生成分析报告框架

- 基于已获取的数据，Agent 用 LLM 生成报告结构，其中包括：大盘综述、板块表现、资金流向、技术指标、投资建议等等
- 这一步需要结合用户的偏好，比如是否想要投资建议，还是仅事实数据

4. 调用可视化工具生成图表

- 使用matplotlib、pyecharts或内置的可视化Skill，生成K线图、板块轮动热力图、北向资金流向图
- 而且图表要符合用户的习惯，比如：PNG 或者交互式 HTML

5. 汇总结果并推送

- 将文字报告和图表整合成PDF，或直接生成网页链接
- 然后发送到用户指定的邮箱
- 并在微信或飞书上发送通知：比如：您的A股分析报告已生成，请查收邮箱

上面我们定出智能体 Agent 的规划路径，除此之外，我们还需要定义退出条件。因为 Agent 不能无限执行下去，它需要知道什么时候应该停下来。推出条件有三种情况，分别为：

- 第一：成功退出：当所有步骤都执行完毕，报告已发送，且用户没有进一步追问。
- 第二：异常退出：如果某个关键步骤失败，比如API无法访问，且重试3次后仍失败，Agent 应该终止，记录错误并通知用户。
- 第三种情况：如果用户中途说 ‘算了，不用了’，Agent 也应该可以立即停止并清理资源。

有了明确的退出条件，智能体 Agent 才能做到 ‘召之即来，挥之即去’，不会在后台偷偷运行。

《翻页》

【小结：三维驱动让Agent从“听话”到“懂事”】（9:00-10:00）

智能体 Agent 任务的三维驱动框架我们介绍完了，下面我们再总结下：

- 第一层：任务定义层：该层负责把模糊指令变成清晰目标，并明确成功标准
- 第二层：状态感知层：该层负责挖掘用户偏好，感知环境状态，构建执行上下文
- 第三层：流程控制层：该层负责规划可执行路径，并动态调整，以及定义退出条件。

正是这种‘定义-感知-控制’的循环，让智能体 Agent 能够处理那些看似模糊、复杂的现实任务。它不再是简单执行命令，而是像一个成熟的助理——先问清楚，再看环境，最后动手。

上面我们讲了 Agent 如何通过定义、感知、控制这三层解构用户下达的任务。但任务拆解完之后，Agent 具体是怎么一步步执行的呢？它的大脑是如何运作的呢？

《翻页》

3.2.2 Agent 任务执行（ReAct 范式）：思考-行动-观察的循环

【开场：引入ReAct】（0:00-1:00）

这就是接下来我们要讨论的Agent最基础的执行模式：ReAct范式。

ReAct 是 Reason + Action 单词的缩写，它具有两个含义，那就是：思考+行动。

这个名字最早来自2022年Google和Princeton的一篇论文，但到今天，它已经成为所有 Agent 框架的底层基因。

简单来说，ReAct 就是一个循环：

《翻页》

思考 → 行动 → 观察 → 再思考 → 再行动 → ……

Agent 的这种执行模式非常类似人做一件事：首先想一下怎么做，然后动手，看看结果怎么样，再想下一步。

如果大家不理解也没有关系，下面我们就用一个具体的例子来走一遍这个循环。

《翻页》

【案例：股票分析的ReAct循环】（1:00-4:00）

还是那个用户需求：‘帮我分析一下最近的股票市场’。

下面我们看看Agent内部会发生什么。

《翻页》

第一轮：

模型收到用户需求：

代码块

- 1 模型先进行思考：Thought：用户想分析股票市场，但‘股票市场’太宽泛了。我需要先确定具体范围，否则后续操作没有方向。
- 2 然后模型采取行动：Action：执行引擎调用澄清函数，向用户反向提问：“您想分析哪个市场？A股、港股还是美股？关注哪些板块？”
- 3 最后模型观察，也就是 Observation 到用户的后续回复：“分析A股的新能源板块。”

于是，第一轮结束。

Agent 通过思考，结果发现自己信息不足，于是采取行动——提问，并持续观察直到用户的明确回复。

然后 ReAct 接着进入到了第二轮。

《翻页》

第二轮智能体 Agent 确认了用户需求的具体范围：

代码块

- 1 Thought:得出结论，用户希望分析的是A股新能源板块。然后智能体开始获取这个板块的行情数据。
- 2 Action：它准备调用股市提供的 API，并传递参数：market=‘A-share’（市场是A股），sector=‘新能源’（板块是新能源）
- 3 然后智能体 Agent 再此进入Observation阶段，直到获取到最新行情数据（个股涨跌幅、板块指数、成交量等）

我们发现第二轮 Agent 思考后决定调用API，并最终拿到了数据。

《翻页》

接着智能体 Agent 执行进入到了第三轮：

1. 智能体思考Thought：数据已经就绪，接下来需要生成分析报告了。但用户并没有指定具体格式，是用纯文本还是用图表呢？根据历史偏好，他比较喜欢图表+摘要。那我就调用报告生成工具，并指定包含图表。
2. 思考完毕，智能体进入到Action阶段：它会调用 `report_generator` 函数，并传递参数：`include_charts=True, format='PDF'`
3. 最后智能体进入到观察Observation阶段：直到报告生成成功，并保存为PDF文件。

《翻页》

接着智能体 Agent 执行进入到第四轮：

代码块

1. 智能体接着Thought：报告已生成，需要推送给用户。他之前要求推送到邮箱和微信。
2. 思考完毕进入Action阶段：它会调用 `send_email` 和 `send_weixin_message`
3. 最后智能体进入到观察Observation阶段：直到信息发送成功。

《翻页》

接着智能体 Agent 执行进入到第五轮：

代码块

1. 智能体最后Thought发现：所有任务都已经完成，可以结束了。
2. 然后进入Action阶段：发现已经无事可做，返回最终确认消息

最终这个智能体就执行结束了。

这就是一个完整的智能体 ReAct 循环。我们发现每一步都是‘思考-行动-观察’，它们一环接一环，并环环相扣。

《翻页》

【ReAct三步深度拆解】（4:00-6:30）

我们再来拆解一下这个循环的三个核心步骤。

第一步：Thought（思考）

这一步是ReAct的灵魂。Thought不是简单的if-else判断，而是大模型基于当前状态生成的自然语言推理。

比如第一轮Thought：‘用户想分析股票市场，但‘股票市场’这里指代实在是太宽泛了。’

这是模型理解了用户意图后，结合常识得出的结论。我们并没有给模型预设任何规则，纯粹是它自己推理出来的。

这里Thought的内容通常包括：

- 对当前状态的评估（比如：‘我已经知道什么？’）
- 对下一步行动的推理（比如：‘接下来我应该做什么？’）
- 对可能结果的预测（比如：‘如果调用这个API，可能会得到什么？’）

第二步：Action（行动）

行动是Thought的具体落地。它可以是：

- 调用一个工具（比如：API、方法函数等等）
- 或者向用户提问
- 或者执行一段代码
- 甚至只是更新内部状态

第三步：Observation（观察）

观察是对上一步行动的反馈。可能是：

- 工具返回的数据（比如：调用API返回的JSON）
- 用户的新消息
- 代码执行的输出
- 或者错误信息（比如：调用API超时）

观察结果会被追加到对话的历史中，并作为下一轮思考的输入。

整个 ReAct 循环会一直持续，直到满足某个退出条件——比如任务完成、用户终止、或者达到最大步数限制等等。

这里的关键点在于：每一轮的Thought都是基于当前完整上下文实时生成的，而不是事先写好的剧本。

这也就是 ReAct 能处理未知情况的原因和魔法所在。

【为什么不是简单的if-then?】（6:30-8:00）

也许有同学会认为：‘这不就是循环吗？和传统编程里的if-while循环有什么区别？’

——好问题！我们来看一个对比。

《翻页》

这就是传统编程的 if else：

代码块

```
1  if user_input contains 'A股':
2      call_a_stock_api('A-share')
3  elif user_input contains '港股':
4      call_a_stock_api('HK')
5  else:
6      ask_for_clarification()
```

我们发现这是写死的规则。

如果用户说‘我想看看最近涨得好的板块’，这个规则就失效了——因为它并没有包含‘涨得好’这个条件。

而且现实情况是，我们根本无法根据条件穷举用户的所有提问，我们不是用户肚里的蛔虫，也不能将读心术作用于每个用户。

《翻页》

我们再来看下ReAct模式：

当模型看到‘我想看看最近涨得好的板块’，此时它会进入思考阶段：

‘涨得好’可能意味着涨幅排名靠前。我需要获取板块涨幅榜数据。但用户并没说是哪个市场，我得先问一下，或者默认A股。

然后模型可能先获取A股板块的涨幅榜，如果用户没指定市场，再问。

我们发现整个过程并没有预设规则，完全是由模型根据语义理解，并动态生成。

这种模式的结果就是，它能处理无数种未曾见过的人类表达方式。

《翻页》

我们再来总结下传统模式和 ReAct 模式的本质区别：

- 传统编程模式下：规则是有限的，由开发者预先定义。
- ReAct模式下：推理是无限的，由模型实时生成。

它们两者的关系我们来打个比方：

传统编程就像是一本菜谱，你只能做上面写的菜

ReAct 更像是一个会做饭的人，你告诉他‘想吃点清淡的’，他能根据现有食材和厨艺自由发挥。

声明：我这里不是刻意说 ReAct 行为模式就一定就是最好的，宇宙第一的，有时严肃的生产级场景，基于规则和可预期的行为模式会更合适一些。两者相得益彰，各自精彩，作为技术人员，大家一定要学会灰度管理，而不是简单非0即1的二维是非观。我们对待技术不要用宗教态度面对，而更应该采取中立的态度。做技术应该是像渣男而不是像纯爱战士，哪种技术对我们有利我们就用哪个，不用忠诚，当然这个观点大家批判继承。

《翻页》

【ReAct的局限性与改进方向】（8:00-9:30）

下面，我们就来举几个反例，说一下 ReAct 不是万能的。它有几个天然局限：

第一，可能陷入死循环。如果思考总是无法导向行动，或者行动总是失败，Agent可能会在原地打转。比如，API一直超时，它可能反复尝试，不知道换条路。

第二，推理深度有限。对于一些需要多步复杂推理的任务（比如数学证明），简单的ReAct可能不够，需要更结构化的思维链（Chain-of-Thought）或者思维树（Tree-of-Thoughts）。

第三，没有自我纠错机制。ReAct 的‘观察’只是反馈，但如果观察结果明显不合理（比如API返回空数据），Agent不会主动反思‘是不是我理解错了？’——它只会拿着空数据继续下一步。

这就引出了ReAct的进化版本：带反思的ReAct。在思考之前，Agent会先反思上一轮的行动是否合理，如果发现错误，就调整策略。

比如股票分析中，如果API返回‘新能源板块暂无数据’，反思机制会触发：

‘为什么没数据？可能是API参数错了，或者今天休市。我需要换个数据源，或者告诉用户今天休市，建议查看历史数据等等。’

这种反思能力，可以让 Agent 从‘按部就班’进化到‘主动纠错’。

这些内容我们会在以后的课程中再详细讲述。

《翻页》

【小结与过渡】（9:30-10:00）

上面我们深入了 Agent 的任务执行模式：ReAct 范式——思考、行动、观察的循环。

它的本质核心就三句话：

- 第一：思考：基于当前状态推理下一步

- 第二：行动：调用工具或反向提问
- 第三：观察：获取反馈，更新状态。

我们也知道了，这个过程并不是简单的 if-then，而是大模型驱动的动态决策。

也正是有了这种机制，Agent 可以具备能够应对开放世界的无限可能性。

同时我们也知道了 ReAct 还不够完美——它需要反思能力来纠错，需要记忆来保持长期一致性。

接下来，我们就要进入更高级的话题：Agent 的自我反思与偏差修正。

《翻页》

3.3 反思改进：Agent 的自我纠错机制（5分钟）

【开场：从ReAct到反思】（0:00-1:00）

上面我们介绍了 ReAct 范式——思考、行动、观察的循环。这个循环可以让 Agent 能够动态决策，应对各种未知情况。

但大家有没有想过一个问题：如果 Agent 思考出错了呢？比如它调用了错误的API，或者误解了用户意图，怎么办？

在 ReAct 基础版本里，它只会拿着错误的结果继续往下走——一步错，步步错。

这就像一个人埋头赶路，即使走错了方向也不知道回头。

所以，真正的智能体需要具备一个关键能力：自我纠错。

下面我们就来具体讲述 Agent 的反思改进机制。

《翻页》

【自主迭代循环：代码层面的反思】（1:00-2:30）

我们先看一个核心概念——自主迭代循环。

我们从代码层面看下 Agent 反思机制的代码级表达：

代码块

```
1 while not task_complete:
2     result = agent.execute(task)
3     is_passed, feedback = audit_agent.verify(result, standard)
4     if not is_passed:
```

这段代码看起来很简洁，但背后却大有深意。

- 第二行：`agent.execute(task)`：意思是：执行 Agent 会执行当前任务，生成一个结果。
- 第三行：`audit_agent.verify(result, standard)`：另外一个专门的‘审计 Agent’，会检查上一个 agent 执行结果是否符合预期标准。
- 如果不符合，就把审计意见追加到任务描述中，重新让执行 Agent 带着审计 Agent 的反馈结果继续执行。
- 这个循环过程会一直持续，直到等到审计 Agent 最终通过审计，或者达到其他终止条件，比如说设置最大循环次数。

自主迭代循环这个模式模仿了人类的工作方式。

比如：写一篇文章，自己检查一遍，发现问题，修改，然后再检查，直到最终满意。

再回到最初股票分析的例子：假设 Agent 的任务是‘生成一份新能源板块分析报告’，第一次执行生成了一个纯文本报告。审计 Agent 一看标准：‘不对，报告中必须包含图表’，于是给出反馈：‘缺少图表，请补充’。

新任务被进一步更新为‘生成一份新能源板块分析报告，反思意见：缺少图表，请补充’。

执行 Agent 第二次执行时就会记得加上图表这个审计反思要求。

这个过程就是最简单的循环反思闭环过程。

这个循环背后其实有三个关键机制，让整个反思可以真正地落地。

《翻页》

【关键机制一：智能拦截】（2:30-3:30）

第一个机制就是**智能拦截**。

当审计 Agent 发现结果不符合预期时，它不是简单地抛出一个错误，而是拦截执行流程，并把反馈注入到历史上下文中。

注意‘注入’这个词。它不是覆盖原来的任务，而是在原有任务基础上追加反思意见。这样执行 Agent 最终就能理解：‘哦，原来是我上次做错了，审计 Agent 告诉我问题在这儿，这次我要改’。

而且拦截的时机也非常重要。有些系统是细粒度拦截，比如在每个步骤后都拦截检查点，而有些系统是在整个任务完成后统一检查，比较粗粒度。具体用哪种拦截方式，取决于任务的复杂度以及对准确性的要求。

智能拦截的核心价值在于：防止错误结果被当作下一步的输入。

智能拦截机制就像工厂里的质检员，一旦发现次品，立刻把它从生产线上拿走，不让它进入下一道工序。

《翻页》

【关键机制二：状态快照】（3:30-4:15）

下面我们再看第二个机制：**状态快照**。

程序每次循环迭代，Agent 都会保存完整的执行状态——包括当前的任务描述、已经完成的部分、中间结果、甚至Agent 的思考历史。

为什么需要这个？因为反思后重新执行时，Agent 并不需要从头再来。它可以基于快照，只修改有问题的部分。比如在生成报告的例子中，第一次执行已经获取了数据，只是没生成图表。第二次执行时，它可以复用之前的数据，只补充图表部分，而不是重新调用 API 获取数据。

这个过程防止重复劳动。

状态快照的机制让反思的开销变得很小，否则每次反思都需要重跑所有步骤，成本太高了。

这就好比写论文，导师让你修改其中某一段。你不会重写整篇论文，而是只修改老师让你修改的那一段，然后保留其他部分。

状态快照就是让Agent也能‘只修改一段’。

《翻页》

【关键机制三：收敛检测】（4:15-4:45）

Agent 的自我纠错的第三个机制就是**收敛检测**。

反思循环有可能会陷入死循环——Agent改了一次，审计还不满意，再改，还是不满意……如果一直不满意，就会无限循环下去。

收敛检测就是为了防止这种情况发生。

它会在每次迭代后比较这次和上次的输出，如果连续多次输出的差异小于某个阈值，就认为已经无法进一步改进，强制终止循环。

比如，Agent 生成的报告和上一版只有几个标点符号不同，审计依然不满意，但 Agent 已经不知道还能怎么改了。这时就应该停止，避免浪费算力。

这个阈值可以根据任务类型动态调整。比如创意类任务可能允许较大差异，而事实类任务要求严格一致。

《翻页》

【小结：反思让Agent更可靠】（4:45-5:00）

上面我们介绍了让 Agent 自我纠错的方法，下面我们再快速总结一下反思改进的这三个关键机制：

- 第一：智能拦截：发现问题时拦截流程，注入反馈；
- 第二：状态快照：保存中间状态，避免重复劳动；
- 第三：收敛检测：防止无限循环，及时止损。

有了这些机制，Agent 就不再是‘一条路走到黑’的莽夫，而是一个能自我修正的智者。

在复杂任务中，反思能力往往决定了 Agent 的可靠性。

《翻页》

第四部分：趋势演进——从API到Agent的必然之路（20分钟）

【开场：从“手工时代”到“智能时代”】（0:00-1:00）

前面我们花了大量篇幅拆解 Agent 的内部机制——公式、任务解构、ReAct模式、以及执行后的反思。

这些内容很硬核，但大家有没有想过一个问题：Agent 这些能力是怎么一步步演化出来的？

为什么2022年不提 Agent，2023年也没有提，偏偏是2025-2026年 Agent 突然就爆了？

下面，我们就来梳理一条清晰的技术演进路线图。

《翻页》

这条路线共分为四个阶段，就像从‘石器时代’到‘工业革命’的跃迁。

理解了它，你就知道我们今天站在哪里，以及明天要去哪里。

这四个发展阶段依次是：

代码块

- 1 阶段一：基础API调用（2022-2023）
- 2 阶段二：提示词工程（2023-2024）
- 3 阶段三：Function Calling（2024-2025）
- 4 阶段四：Agent框架（2025-2026）

每个阶段都不是孤立发生的，而是在解决上一个阶段的痛点上持续演进的。

下面我们亲历这些阶段。

《翻页》

【阶段一：基础API调用——开发者是“手工编排者”】（1:00-2:30）

第一个阶段：基础API调用，时间发生在 2022-2023。

这个时期，大模型刚刚出圈不久，大家发现可以用API调用它。

但当时的使用方式非常原始：开发者手动编排，在程序里直接通过 API 调用模型，而且每次的调用都独立。

比如，你想做一个能查天气、能订机票的对话机器人。你需要写一大段代码：

代码块

```
1  # 伪代码
2  user_input = get_user_input()
3  if '天气' in user_input:
4      city = extract_city(user_input)
5      weather_data = call_weather_api(city)
6      response = generate_weather_text(weather_data)
7  elif '机票' in user_input:
8      # 又是一堆逻辑
9      ...
10 else:
11     response = call_llm_api(user_input) # 普通对话
```

看到了吗？所有的逻辑都要开发者自己写。意图识别（也就是 if-else）、实体抽取（比如这里的 extract_city 函数）、API调用（call_weather_api），全是硬编码。

这个阶段的核心特征就是：

- 模型只负责生成文本，其他全靠开发者写代码。
- 每次调用都是独立的，模型没有记忆，对话历史要开发者自己维护。
- 扩展性极差：每新增一个功能，就要改代码。

这个阶段，AI更像一个‘文本生成器’，而不是智能助手。开发者是那个在后台默默编排一切的‘手工匠人’。

这个阶段优点是生成的内容拟人化。

痛点就是：太累！每增加一个工具，就要写一堆胶水代码。而且模型根本不理解自己在做什么，只是被动输出。

《翻页》

【阶段二：提示词工程——让模型“听话”】（2:30-4:00）

第二个阶段：提示词工程阶段，它发生在 2023-2024 之间。

开发者们发现，与其写一堆if-else，不如在Prompt里告诉模型该怎么做。于是，提示词工程诞生了。

比如：我们可以在Prompt里写：

代码块

- 1 你是一个智能助手。如果用户问天气，请按格式输出：{"intent": "weather", "city": "城市名"}
- 2 如果用户问机票，请输出：{"intent": "flight", "from": "出发地", "to": "目的地"}
- 3 其他情况正常对话。

然后，你把用户输入和这个Prompt一起发给模型。模型就会按照你指定的格式输出JSON。

然后你在代码中解析这个JSON，并调用对应的API。

第二阶段的进步在于以下三点：

- 第一：模型开始输出结构化数据，而不是纯文本。
- 第二：意图识别和实体抽取从代码转移到了Prompt，减少了硬编码。
- 第三：开发者只需要维护Prompt，而不是大量if-else。

但问题依然存在：

- 第一：仍需人工介入：输出格式是开发者规定的，模型只是‘填空’。
- 第二：Prompt工程复杂：要不断调优，否则模型输出格式可能出错。
- 第三：模型依然没有‘自主选择权’：它只能按Prompt的指令输出，不能自己决定调用哪个工具。

在这个阶段，AI像是被绑住手脚的演员，你告诉它‘说这句台词’，它就重复。

虽然比第一阶段灵活，但本质还是被动的。

《翻页》

【阶段三：Function Calling——模型学会“动手”】（4:00-5:30）

下面发展到第三个阶段，也就是 Function Calling 阶段，时间大概是 2024-2025 之间。

2024年，OpenAI 扔出一颗核弹：Function Calling。这个功能彻底改变了游戏规则。

什么是Function Calling?

简单来说，就是开发者可以给模型提供一系列函数的描述（包括函数名、参数、功能说明），模型在理解用户提交的函数描述后，自主决定调用哪个函数，并生成相应的调用参数。

我们来看一个简化版的函数描述：

代码块

```
1  {
2    "name": "get_weather",
3    "description": "获取指定城市的天气",
4    "parameters": {
5      "type": "object",
6      "properties": {
7        "city": {"type": "string", "description": "城市名"}
8      },
9      "required": ["city"]
10   }
11 }
```

当用户问‘北京天气怎么样’，并把上面的函数描述连同用户问题作为提示词的一部分通过调用模型 API 的时候，模型输出类似下面的回复：

代码块

```
1  {
2    "function_call": {
3      "name": "get_weather",
4      "arguments": "{\"city\": \"北京\"}"
5    }
6  }
```

你的代码收到这个回复，然后就去调用真实的天气API，然后再把天气 API 返回的结果传给模型，模型再生成自然语言回复。

第三阶段的核心突破主要有三个：

- 第一：模型开始自主选择函数，而不是按Prompt填空。
- 第二：自然语言到结构化执行的范式转换——这是Agent能够‘动手’的技术前提。
- 第三：开发者只需要函数定义描述，剩下的交给模型。

为什么说 Function Calling 具备划时代的革命性呢？

因为在此之前，模型只能‘说话’；Function Calling 第一次让模型学会了‘发号施令’。

它输出的不再仅仅是文本，而是可执行的指令。

这就像一个人终于从‘纸上谈兵’变成了‘指挥官’——虽然还不能亲自执行，但已经能精准调度资源了。

但必须澄清，这时的 Function Calling 还不是完整的Agent，因为它没有记忆，没有规划，没有反思。

它只是一个‘会调用工具的模型’，而不是一个‘能持续执行任务的智能体’。

《翻页》

【阶段四：Agent框架——完整闭环】（5:30-7:00）

最后一个阶段，也就是第四个阶段：Agent 框架阶段，时间是 2025-2026 年。

Function Calling 铺平了道路，但真正的智能体还需要更多组件。于是，Agent 框架应运而生。

Agent框架 = 规划（Planning）+ 记忆（Memory）+ 工具调用（Tool Use）+ 反思（Reflection），再加上底层的LLM。

这就是我们之前讲的完整公式。

在这个阶段，Agent不再是单次调用，而是可以持续执行多步任务。它自己规划步骤，自己调用工具，自己检查结果，错了就反思重试，直到任务完成。

这里举一个典型的Agent执行流程的例子：

1. 用户说：‘帮我策划明天的出差行程’
2. Agent 先进行规划，规划结果是需要查天气、查航班、订酒店、创建日历、约车
3. 于是第一步：agent 调用天气API，底层用的是 Function Calling 机制
4. 然后是第二步：根据天气推荐着装，这步是由 LLM 生成
5. 接着是第三步：agent 接着调用航班API，底层用的是 Function Calling 机制
6. 然后是第四步：如果航班没票，进行反思。是不是日期不对？改查高铁
7. 接着是第五步：创建日历事件，调用日历 API
8. 然后是第六步：约车，调用打车 API
9. 最后是第七步：汇总结果，推送用户

整个过程，Agent 都是进行自主决策、自主执行、自主纠错。

开发者只需要定义好工具和初始指令，剩下的托管给 Agent。

在第四个阶段，产生了很多代表性的智能体 Agent 框架，比如：

- LangGraph：这是基于图的多智能体协作
- AutoGen：这是微软的多智能体对话框架
- CrewAI：这是角色化多智能体协作

等等，正是这些框架的不停进化，从而让 Agent 开发真正从‘手工作坊’走向‘工业流水线’。

《翻页》

【Function Calling的再强调：范式转换的支点】（7:00-7:45）

让我们回顾这四个阶段，我们可以清晰地看到一条AI发展的主线，那就是AI从‘被动输出’走向‘主动执行’。

而这一切的转折点，就是 Function Calling。没有它，Agent 永远只能‘建议’而不能‘动手’。

为什么说 Function Calling 这么关键？因为它是从自然语言到结构化执行的桥梁。

在此之前，人和AI的交互停留在‘我说你听’；在此之后，人和AI的交互变成了‘我下指令，你执行’。

就像互联网的诞生让信息自由流动一样，Function Calling 让行动指令自由流动。

它是Agent世界的‘HTTP协议’，是一切上层建筑的基石。

所以，当我们今天谈论 Agent 时，一定要记住：没有Function Calling，就没有Agent的今天。

Function Calling 后期我们也会专门开辟一个课程详细讲述，大家即使这里不清楚也没有关系，后期我们会重点讲。

《翻页》

【小结与过渡】（7:45-8:00）

好，我们再来快速回顾并加深一下智能体从诞生到成熟的技术演进四个阶段的印象：

- **第一个阶段：基础API调用：**开发者手工编排，模型只是文本生成器
- **第二个阶段：提示词工程：**用Prompt引导结构化输出，但仍需人工
- **第三个阶段：function Calling：**模型自主选择函数，开始实现从自然语言到结构化执行的过渡
- **第四个阶段：Agent 框架：agent 进化成由规划到记忆再到工具再到反思，最终实现完整闭环**

这四个阶段并不是截然分开的，它们层层叠加并实现螺旋上升演进，最终形成了我们今天看到的智能体生态。

《翻页》

4.2 架构演进：从单体到群体（7分钟）

【开场：从“单打独斗”到“团队作战”】（0:00-1:00）

上面我们梳理了智能体 Agent 发展的四个阶段，最后一步是 Agent 框架。

但框架本身也在进化——从最初的‘一个Agent包打天下’，到现在的‘多个Agent各司其职’。

这就像人类社会的演进：原始社会一个人要同时做猎人、采集者、炊事员。

现代社会有了分工，有人专攻代码，有人擅长设计，有人负责审核等等。

接下来我们就来再聊聊Agent架构的演进：从单体到群体。

《翻页》

【单体Agent时代：一个Agent包打天下】（1:00-3:00）

先来看下单体 Agent 时代。这个阶段的典型特征是：一个Agent负责所有事情。

你给它一个目标，它自己规划、自己调用工具、自己反思、自己输出。

听起来很完美，对吧？就像一个全能型员工，既能写代码，又能做设计，还能写文案，很全栈。

优点很明显：简单直接，体现在以下三方面。

- 第一：架构简单：只有一个Agent，不需要考虑多Agent通信。
- 第二：部署容易：一个服务，一个API密钥，跑起来就行。
- 第三：对于简单任务，效率很高。

但有没有缺点呢？

有。

而且缺点同样致命：复杂任务容易‘逻辑断裂’。

什么叫逻辑断裂？

我举个例子：假设你让一个单体 Agent ‘策划一场新品发布会’。它需要同时处理：

- 场地调研（涉及：搜索能力）
- 预算计算（涉及：数学能力）
- 嘉宾邀请名单（涉及：社交网络分析）
- 活动流程设计（涉及：创意能力）
- 宣传文案撰写（涉及：写作能力）
- 风险预案（涉及：推理能力）

这么多不同类型的任务，我们都把它挤在一个Agent的‘大脑’里，这很容易串台。

它在算预算时可能忘了场地的大小，在写文案时可能忘了预算的限制。

就像一个人同时做十件事，最后每件事都做得不完美。

更重要的是，单体 Agent 缺乏专业深度。

一个模型再强，也不可能在所有领域都达到专家水平。让它写代码可能不错，但让它画设计图就力不从心了。

所以，行业开始思考：能不能让多个 Agent 分工协作，就像人类团队一样？

《翻页》

【多Agent系统时代：分工协作的智慧】（3:00-5:00）

后来就进入到了多 Agent 系统时代。

这个时代核心理念就是：让专业的 Agent 做专业的事。

一个典型的多 Agent 系统通常包含三类角色：

第一类角色是：指挥中枢（Commander），主要：

- 负责任务分解与调度
- 它不直接执行具体任务，而是像一个项目经理，把大目标拆成小任务，分配给合适的专家 Agent 去执行，并协调这些专家 Agent 之间的沟通。
- 当专家 Agent 返回结果时，指挥中枢负责整合、并判断是否重新分配。

第二类角色是：专家Agent（Expert），这类 Agent 主要：

- 各司其职，每个 Agent 只负责一个专业领域。
- 比如代码 Agent、搜索 Agent、绘图 Agent、数据分析 Agent 等等。
- 它们在自己的领域内深度优化，可以调用专门的工具和模型。

第三类角色是：审计Agent（Critic），这类 Agent 主要：

- 负责合规性审核和质量把关。
- 它会检查专家Agent的输出是否符合要求，有没有敏感内容，有没有逻辑错误。
- 当发现问题时，它会把反馈发回给指挥中枢，指挥中枢再决定是让原专家 Agent 修改，还是分配给其他专家。

这个结构是不是很像一个成熟的公司？

CEO（也就是指挥中枢）定战略，各部门（也就是专家Agent）负责执行，质检部（也就是审计Agent）把关。

这种分工协作的架构，解决了单体 Agent 带来的两大痛点：

1. 第一就是：专业深度：每个专家 Agent 都可以针对特定任务微调，甚至使用专门的模型。
2. 第二就是：逻辑清晰：任务被拆解后，每个步骤由最合适的角色完成，减少串台。

而且，审计Agent的存在让系统有了多层质量保障——不仅每个专家自己会反思，还有独立的第三方检查，大大提高了输出的可靠性。

《翻页》

【案例：营销方案生成的全流程】（5:00-6:30）

下面我们用一个真实营销方案案例，看看多 Agent 系统是如何协作的。

我们有一个客户，他们营销部目标是：‘为一款新上市的智能眼镜制定营销方案’。

这个任务涉及很多道工序，比如：市场调研、策略定价、创意设计以及渠道分发。

技术同学经过分析后，提出解决方案。通过设计多个 Agent（一共六个 Agent）来实现。

这六个 Agent 角色如下：

第一个 Agent——也就是指挥中枢 Agent：它的作用是拆解任务

指挥中枢 Agent 会确定整个营销任务。该任务需要四步：研究竞品、制定策略、生成创意、分发执行。

第二个 Agent——研究 Agent

- 该Agent任务：全网抓取竞品数据。
- 它会爬取各大电商平台、科技媒体、社交媒体，收集同类产品的价格、卖点、用户评价等等。
- 然后它会输出一份《竞品分析报告》，该报告会包含价格、用户痛点、市场空白点等等。

第三个 Agent——策略 Agent

- 该Agent任务：基于竞品报告，生成SKU定位与定价模型。
- 它会结合成本数据、目标用户画像，给出建议：‘比如：定价1999元，主打运动健康监测，差异化卖点是长续航+AI教练’。
- 策略 Agent 最终会输出《产品定位与定价策略文档》。

第四个 Agent：创意Agent登场

- 该Agent任务：根据策略文档，自动生成广告海报和短视频脚本。
- 它会调用绘图模型生成3-5版海报，用视频生成模型制作15秒短视频。
- 最终输出创意素材包。

第五个 Agent——分发Agent

- 该Agent任务：选择最佳时段在各平台发布。
- 它分析各平台用户活跃时间，制定发布计划：抖音晚8点、微博上午10点、小红书周末中午等等。
- 然后分发 Agent 会自动调用各平台的API完成最终发布。

最后是第六个 Agent——审计Agent。这个 Agent 会实时监控，并贯穿全流程：

- 它会检查研究 Agent 的数据源是否可靠
- 检查策略 Agent 的定价是否合理（比如是否低于成本）
- 检查创意 Agent 的素材是否有版权风险
- 检查分发 Agent 的排期是否符合平台规则
- 发现问题立即反馈给指挥中枢 Agent，让指挥中枢 Agent 进行协调和修正

在这个真实案例的 Agent 全流程中，每个Agent只做自己最擅长的事，审计Agent全程把关，指挥中枢确保步调一致。

最终我们测试结果，发现它的质量和效率都远超单体Agent。

《翻页》

【小结：从单体到群体的必然演进】（6:30-7:00）

Agent 架构的发展我们就简单介绍完了，下面我们再简单总结下，其实就两条：

- 第一条：单体Agent时代：该时代，Agent 优点简单直接，缺点是复杂任务容易逻辑断裂，缺乏专业深度。
- 第二条：多Agent系统时代：在这个时代，强调分工协作，表现形式是：指挥中枢 Agent + 专家 Agent + 审计Agent，大家各司其职，多层质保。

智能体 Agent 这种架构的演进并非偶然，而是技术发展的必然。

因为当任务复杂度超过单智能体能力的上限，群体智能就成了唯一解决方案。

今天，多 Agent 系统已经广泛应用于在金融分析、医疗诊断、乃至软件开发等各个领域。

在我们的课程中，我们也都会深入介绍，其中 OpenClaw、AutoGen、CrewAI 这些优秀的智能体框架也都内置了多 Agent 的协作能力，后续我们会让你亲手实战进行体验。

《翻页》

4.3 工程协议的出现：MCP与A2A（5分钟）

【开场：当Agent越来越多，问题来了】（0:00-1:00）

上面我们讲了智能体 Agent 从单体到群体的架构演进。

我们发现多个 Agent 分工协作，就像一支专业团队。

看起来格外美好，对不对？

但这里会冒出一个新问题。在这支 Agent 团队里，Agent 之间怎么进行互相沟通？怎么调用外部工具？如果 Agent 是用不同框架开发，它们能一起工作吗？

这就引出了新的痛点课题：**工程协议**。

在人类世界，如果没有共同语言和通用接口，协作就是空谈。

同样，在 Agent 世界，也需要一套标准化的协议来支撑大规模的智能体网络。

下面我们就来介绍两个正在改变游戏规则协议：MCP 和 A2A。

《翻页》

【MCP：模型上下文协议——Agent的“万能插座”】（1:00-2:30）

我们先看 MCP，它的全称是模型上下文协议（Model Context Protocol）。

这个协议要解决什么问题呢？

很简单：决定 Agent 如何**标准化**地连接各种数据源和工具。

在 Agent 开发中，我们经常要接入各种API：查天气的、订机票的、访问数据库的、操作Excel的等等……

其中每个API都有自己的认证方式、参数格式、返回结构。

Agent 每接入一个新工具，就得写一堆适配代码。

这就像你家里有十种电器，每种都配了不同的插座，每次都要找转换头，你说你烦不烦？

MCP的出现，就是定义一套统一的标准接口。

它使得任何数据源或工具，只要实现了MCP协议，Agent 就可以像插USB一样直接使用。

举个真实项目，一个朋友，它的服务客户是金融机构，大家都知道金融场景会比较严肃，我的朋友就是做金融类数据服务的。

他把自己的数据库包装成 MCP 服务器，然后让 Agent 通过MCP客户端发送标准化的查询请求，从 MCP 服务器获取结构化的数据。

这种抽象化的包装完全不用关心底层数据库到底是 MySQL 还是 MongoDB，也不用关心 API 密钥怎么传。

他的这个 MCP 核心组件主要由三部分组成：

1. **第一：MCP服务器**：也就是数据源的提供方，它会按照 MCP 标准暴露接口。
2. **第二：MCP客户端**：也就是 Agent 端，它通过 MCP 协议与服务器进行通信。
3. **第三：MCP协议**：定义了请求/响应格式、认证方式、错误处理等。

自由有了 MCP，Agent 的世界从一个‘孤岛’变成了一个‘插件化的平台’。

这样只要新工具上线后，只需实现 MCP，全球的 Agent 立刻就能用。

它就像 USB 接口统一了电子设备一样，MCP 正在统一 Agent 世界的工具调用。

《翻页》

【A2A：智能体间通信协议——Agent的“通用语言”】（2:30-4:00）

了解完 MCP，我们再来看另外一个协议——A2A。它是智能体之间的通信协议（Agent-to-Agent）。如果说 MCP 是 Agent 和外部工具的‘握手协议’的话，那么 A2A 就是 Agent 和 Agent 之间的‘对话协议’。

在多 Agent 系统中，Agent 们需要交换信息、分配任务、同步状态。

但如果 Agent 是用不同框架开发的（比如一个用 LangGraph，一个用 CrewAI），它们之间怎么聊天呢？各说各话肯定是不行，因为它们属于不同的堂口，谁也不鸟谁。

A2A的作用，就类似话事人。它定义一套通用的消息格式和交互流程。

任何 Agent，只要遵守 A2A 协议，就能与其他 A2A 兼容的 Agent 自由进行沟通。

想象一个场景：你有一个用 LangGraph 开发的研究 Agent，和一个用 AutoGen 开发的创意 Agent。

研究 Agent 发现了市场趋势，想通知创意 Agent 调整方案。

如果没有 A2A，你得写一堆胶水代码来转换消息。有了 A2A，它们可以直接用标准化的消息格式顺畅进行交流：

代码块

```
1  {
2      "protocol": "A2A",
3      "from": "research_agent",
4      "to": "creative_agent",
5      "type": "task_update",
6      "payload":
7      {
8          "task_id": "marketing_001",
9          "new_finding": "竞品降价20%，建议调整定价策略"
10     }
11 }
```

A2A 的核心价值在于：打破框架壁垒，让不同生态的 Agent 能够协同工作。

这就像人类世界用英语作为通用语言，不管你是中国人还是巴西人，都能通过英语进行交流。

《翻页》

【前瞻：Agent工业化的基础设施】（4:00-4:40）

看到这里，想必大家应该能够理解这两个协议的分量了。

- MCP 可以让 Agent 能够标准化连接世界。
- 而 A2A 可以让 Agent 能够标准化地连接彼此（通信、协作）。

这两者结合，就构成了智能体 Agent 网络世界的基础设施层。

就像 HTTP 协议让互联网通信互联互通。

MCP 和 A2A 协议正在做同样的事情：它们可以将数百万个不同框架开发、不同能力的Agent，组成一个庞大的智能体网络。

在这个网络里，可以随时召唤一个擅长数据分析的Agent，又可以调用另一个擅长可视化的Agent，再接入MCP兼容的数据库等等。

所有这一切都是标准化的、即插即用的。

这就是Agent工业化的未来。

它将不再是手工作坊式的单个 Agent 开发，而是像搭积木一样，从全球 Agent 市场中组合出你需要的 Agent 战队。

【小结与预告】（4:40-5:00）

MCP 和 A2A，我们今天只做简单概念介绍。它们的具体实现、如何使用、有哪些坑，都需要深入讲解。

这里预告一下：在后面的课程，我们会带大家手把手实战MCP和A2A。

搭建一个真正的跨协议、跨框架的多Agent系统。

到时候你会看到：Agent如何通过MCP调用外部数据服务，又如何通过A2A与一个Agent共同协作完成任务。

《翻页》

第五部分：实践启航——OpenClaw环境搭建与初体验（30分钟）

5.1 OpenClaw是什么？（5分钟）

【开场：终于要动手了】（0:00-0:30）

下面到了动手环节，君子动嘴也要动手！上面我们一直在讲 Agent 的理论、架构、演进。

讲了大脑、规划、记忆、工具、反思、多智能体……听起来很多概念，即绕又复杂，而且雾里看花。

下面我们需要动手，真正实操下！

而我们这次动手的对象主角，就是目前开源社区最火、GitHub星标破纪录的Agent框架——OpenClaw。

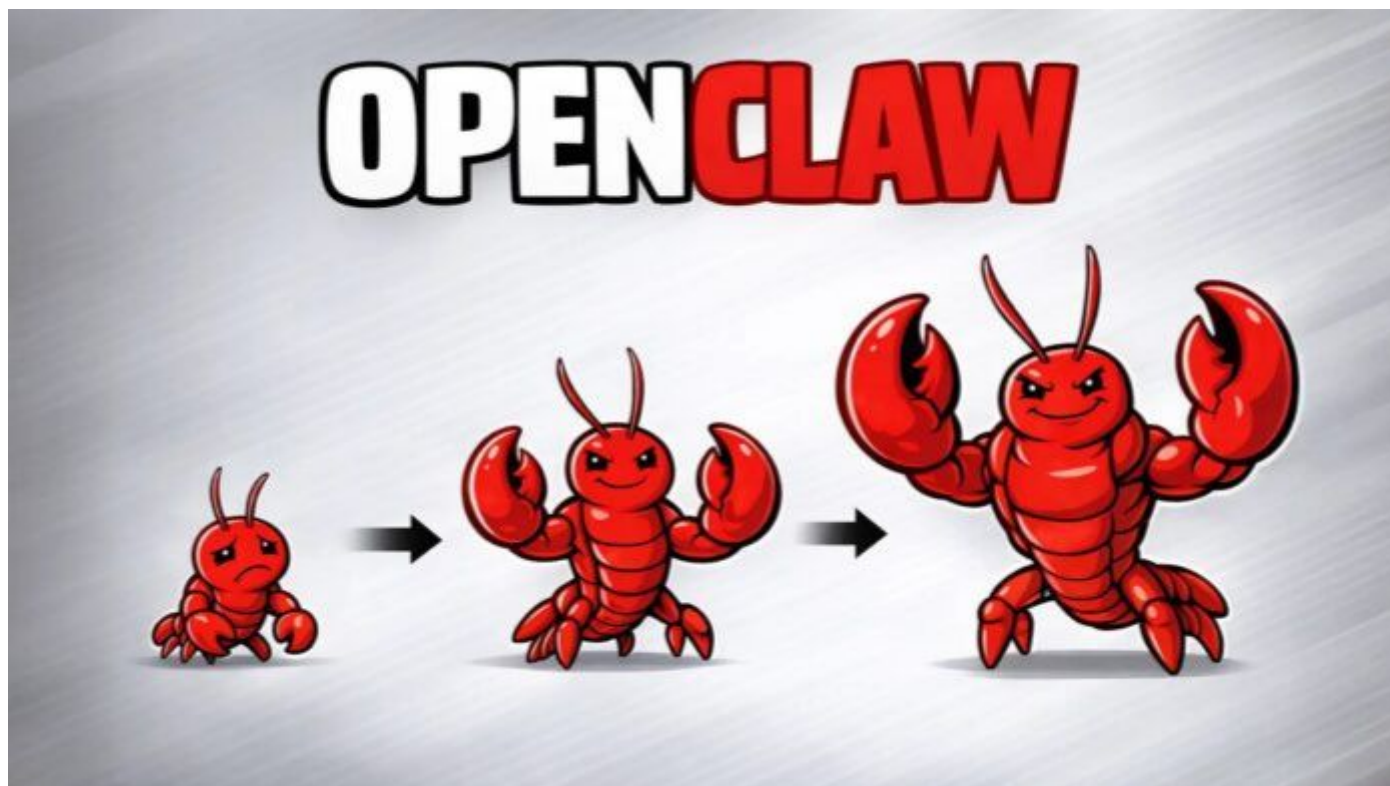
可能你已经听说过它，也可能第一次接触。

这都没关系，下面我会让你彻底明白：

OpenClaw 到底是什么，为什么它能火成这样。

【项目定位】（0:30-1:30）

OpenClaw，也就是我们俗称的——‘龙虾’。



它是一款开源的智能体框架，可以实现把 AI 从「聊天工具」变成「能自主执行任务的数字员工」。

如果你用过 ChatGPT，你会知道它本质上是一个问答系统：你问，chatgpt 回答。

OpenClaw 则不一样。它是一个 AI Agent 平台。

它不仅可以连接20+消息渠道（WhatsApp、Telegram、飞书、钉钉、Discord等）。
它也同样可以主动执行任务、管理你的日程、处理邮件、操作浏览器、调用各种工具等等。
换句话说，ChatGPT是「画饼」，OpenClaw是「真干」。

Openclaw 它的影响力有多大？

截至2026年3月，OpenClaw 在 GitHub 上的星标实现最高纪录。

光这个记录就足够创始人吹牛一辈子了。

【赛博文化现象】（0:30-1:30）

因为吉祥物是龙虾，中文社区将运行 OpenClaw 称为「养虾」，用户自称「养虾人」。

「养龙虾了吗？」成了前几个月AI圈的问候语。

这种有趣的文化标签降低了传播门槛，让一个技术项目有了社交货币属性。

不知道大家是否看到这样的自媒体信息，2026年3月6日，深圳腾讯云总部，近千人排队体验 OpenClaw 安装。

3月8日，深圳龙岗区AI（机器人）局发布了 OpenClaw 使用支持措施的征求意见稿。

一个开源技术项目能引发地方政府的政策关注，这在国内并不多见。

由此足见 OpenClaw 的强大魔力和号召力。

同期，跟龙虾兴起的还有一个智能体社交网络——Moltbook。

它是从 OpenClaw 生态中衍生出来的新型社交平台，跟传统社交平台不同，它是专供 AI Agent 使用的，是智能体之间的社交平台，这听起来一下就有了赛博朋克的感觉。

想想看，数千个 OpenClaw 实例自己在社交网络上发帖、评论、和其他智能体 Agent 讨论哲学问题，这情节想想有点骚浪。

这也许是AI Agent从「工具」走向「社会化存在」的第一个大规模实验场吧，至于后续会发展成什么样子，会不会智能体聊着聊着就产生了自我意识，让我们拭目以待吧。

【热门玩法】（0:30-1:30）

OpenClaw 有多种玩法，分类如下：

1. 赚钱型

- 比如：在 Polymarket 上用AI进行预测市场交易，已有 OpenClaw 月入数万美元的案例
- 比如：ClawWork 项目：「OpenClaw作为你的AI Coworker，11小时赚了\$15K」

2. 生活助手型，openclaw 可以：

- 接管邮件、日历、进行消息管理
- 它也可以自主浏览网页、填表、进行数据抽取
- 同样可以进行文件读写、执行Shell命令等等

3. 社交养成型

- 比如：在 Moltbook 上给 Agent 设定名字和性格，观察其「社交行为」
- 以及 Agent 之间的交互形成了一种「赛博养成」的文化

4. 企业部署型

- 国内养虾用户大量接入飞书、钉钉、企业微信和QQ
- 同时已有专门的插件套件，支持三步 OpenClaw Docker 级部署

所以，不管你未来是想做个人助理、企业自动化、还是复杂的多智能体系统。

OpenClaw 可能都是你绕不开的框架。

OpenClaw 总结下有四个特性。

《翻页》

【核心特性一：自托管——你的数据你做主】（1:30-2:15）

OpenClaw 的第一个核心特性：自托管。

什么意思？就是你可以把它部署在自己的服务器上，数据完全掌握在自己手里。

连接哪个大模型，也由你自己说了算——OpenAI、Claude、Gemini、智谱、月之暗面……

甚至本地跑的小模型，都可以。

这和那些闭源的 Agent 服务有什么区别？区别大了。

闭源服务里，你的对话数据、上传的文件，都可能被用来训练模型，甚至泄露。

而自托管，意味着数据隐私和安全完全由你掌控。

举个例子：如果你是一个金融公司的CTO，要部署一个分析内部报表的Agent，你敢把数据传到第三方吗？肯定不敢。

但用OpenClaw自托管，所有数据都在公司内网，模型也可以选国产合规的，完全放心。

《翻页》

【核心特性二：多模型路由——博采众长】（2:15-3:00）

OpenClaw 的第二个核心特性：多模型路由。

OpenClaw 可以同时接入多个大模型，并且根据任务类型动态选择最合适的模型。比如：

- 针对复杂推理类任务，openclaw 会调用 GPT 或者 Claude
- 针对简单文本处理任务，它会调用国产模型降低成本
- 针对图像生成任务，它会切换到 Stable Diffusion
- 甚至它也可以在不同模型间做负载均衡，防止单个 API 过载。

这个能力，让 OpenClaw 的 Agent 不再局限于某一个模型的‘智商’，而是能博采众长，用最合适的工具处理最合适的问题。

而且，这个能力是自动的——你只需要在配置里定义好模型列表，Openclaw 会根据你的指令和上下文，自己决定用哪个模型。

《翻页》

【核心特性三：技能系统（Skills）——Agent的App Store】（3:00-3:45）

OpenClaw 的第三个特性：技能系统（Skills）。

在 OpenClaw 里，每个工具都被封装成一个‘技能’（Skill）。

你可以把技能想象成一个原子级别的函数功能——比如：查天气是一个技能，发邮件是一个技能，操作 Excel 是一个技能，调用公司内部 API 也是一个技能。

OpenClaw 官方维护了一个技能市场（ClawHub），目前已经有上千个预置技能，一键安装就能用。比如：

- 天气查询技能
- 股票数据技能
- PDF解析技能

- 飞书/钉钉消息推送技能

等等。更厉害的是，你可以自己写你的专属技能，然后发布到ClawHub，让全世界的开发者使用。

这就形成了一个技能生态，从而可以让 OpenClaw 的能力无限扩展。

在后面的实战中，我也会手把手教大家开发自己的技能。

《翻页》

【核心特性四：多平台支持——无处不在的Agent】（3:45-4:15）

OpenClaw 的第四个特性：多平台支持。

OpenClaw不仅仅是一个Python库，它还可以作为服务部署，接入各种聊天平台：

- Slack
- Discord
- Telegram
- 飞书
- 钉钉
- 企业版微信

这意味着，你可以在团队日常使用的聊天工具里，直接@你的openclaw，让它帮你查数据、发通知、甚至执行任务。比如在飞书群里说：‘@财务助手，帮我生成上季度的报销统计表’，Agent 就会自动执行并把结果发回群里。

这种无缝嵌入工作流的能力，让 Agent 不再是独立的‘机器人’，而变成了团队中的一员。

【结尾：为动手做准备】（4:45-5:00）

好了，OpenClaw 核心画像我们已经清晰了：它是一个自托管、多模型自切换、技能化、多平台支持的 Agent 框架，并且恐怖的是，它还在持续进化。

之所以 OpenClaw 能成为最火的框架，得益于它把 Agent 各个组件都做成了灵活可插拔的模块，让开发者可以像搭积木一样构建自己的智能体。

OpenClaw 的架构、设计哲学、记忆、会话、自我扩展、以及我对 MCP 的看法等等，我们以后都会陆续展开讲。

而本节课的任务是，我们先搭建环境，亲手跑起第一个 OpenClaw Agent，让大家有一个更直观的感受先。

好，说到做到，下面我们马上动手。

《翻页》

5.2 环境搭建（15分钟）

OpenClaw 支持从本地到云端多种部署方式。具体选择哪种取决于大家的技术水平、预算和使用场景。

下表列出常见部署模式【备注说明下：根据时间，这个表格内容会有更新】：

部署方式	是否需要网络	所需环境	费用	是否需要 AI 模型	成本	适用人群
本地 npm	-	Node.js 22+	免费	否	低	开发者、macOS/Linux
Docker	-	DockerEngine	免费	否	中	熟悉容器的开发者
阿里云	是	2C2G 40GB	9.9元/月	是（qwen）	极低（3步）	国内首选，新手友好
腾讯云	是	2C2G	~17元/月	否（需购Coding Plan）	极低（3步）	企微/QQ 生态用户
百度云	是	2C4G	0.01元首月	是（千帆模型）	极低（4步）	体验尝鲜，文心生态
华为云	是	Flexus L 实例	~85元/月起	否（需接MaaS）	中等（5步+）	企业用户，合规需求
火山引擎	是	2C4G	9.9元/月	是（方舟模型）	低（3-4步）	飞书用户首选
扣子编程	是	无需服务器	¥49/月起	是（Seed 2.0等）	极低（2步）	零门槛，不想管服务
Railway	是	自动分配	\$5/月免费额度	否	极低（1键）	海外用户，开发者
Zeabur	是	2C4G 专用	按用量计费	是（AI Hub）	极低（模板）	需要多模型 failover

这里有个搭建建议供大家参考：

模型费用其实是大头。由表格可知，服务器成本普遍已降到很低（9.9~99元/年），真正的成本大头在于模型本身的调用成本。

所以选平台时我们要重点看模型套餐价格，而不是只看服务器价格。

这节课，我们采用本地部署的模式，该模式可以运行在你自己的服务器或电脑上，好处是可以做到数据完全自主可控。

《翻页》

步骤1：演示环境说明

该表格是我个人的演示环境，windows操作系统里使用了 WSL2 拉起的 Ubuntu 22.04 操作系统。

Node.js 版本按官方要求要在22以上。

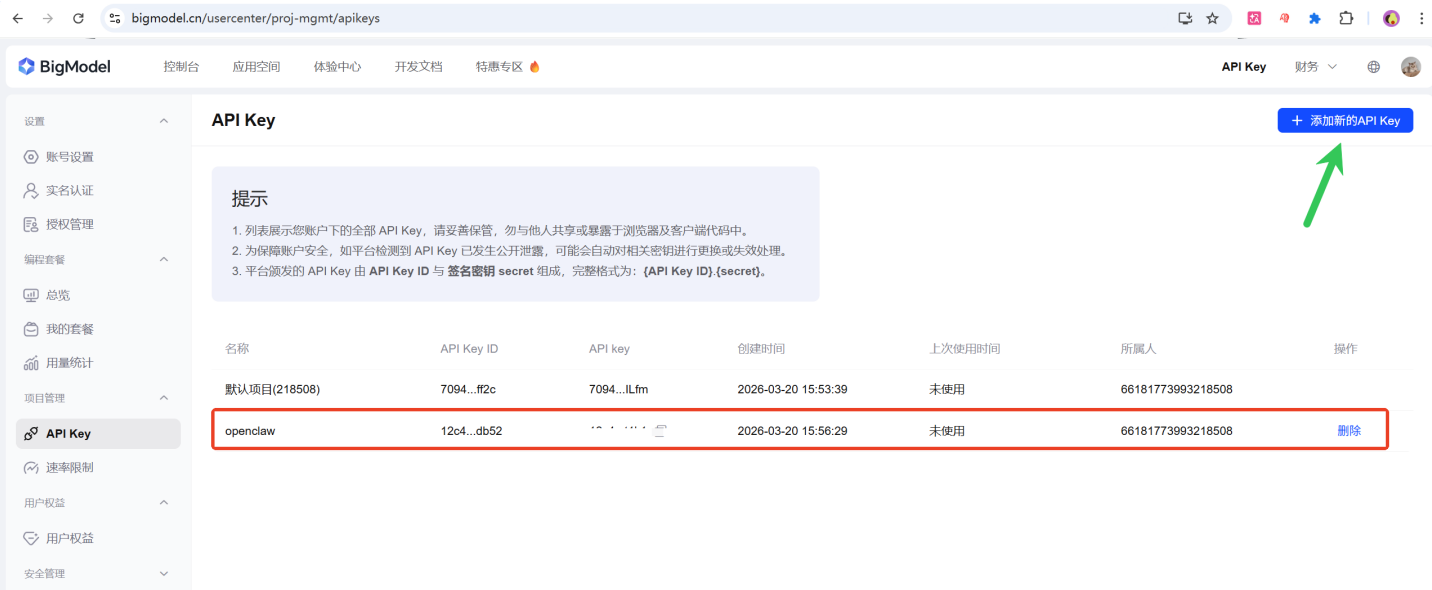
采用的模型大家根据自己的个人情况按需选择即可。

环境	说明
操作系统	windows，强烈推荐 WSL2
Node.js	>= 22（强制要求）
网络环境	需要代理
AI模型	通义千问、OpenAI、Claude、KIMI、智谱 等

我本次演示使用的模式是免费的智谱：GLM-4.7-Flash 模型。

有关 智谱模型 API KEY 免费申请可访问相关网址：<https://bigmodel.cn/usercenter/project-apikeys>

《翻页》



步骤2：安装 Node.js

选好操作系统，并申请大模型 API KEY 之后，我们接着安装 Node.js。

推荐使用 nvm 安装，因为 nvm 可管理多 node.js 版本。安装步骤如下：

《翻页》

代码块

```
1  # 1. 安装 nvm
2  curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
3
4  # 2. 重新加载配置（或重启终端）
5  source ~/.bashrc
```

```
6
7 # 3. 通过 nvm 安装 Node.js 22
8 nvm install 22
9
10 # 4. 设置该 nodejs 版本为默认版本
11 nvm alias default 22
12
13 # 5. 验证
14 node -v # 应显示 v22.x.x
15 npm -v
```

执行成功后截图：

《翻页》

```
|— @excalidraw/mermaid-to-excalidraw@1.1.3
|— corepack@0.31.0
|— node-gyp@11.2.0
=> If you wish to uninstall them at a later point (or re-install them under your
=> `nvm` Nodes), you can remove them from the system Node as follows:

$ nvm use system
$ npm uninstall -g a_module

=> Close and reopen your terminal to start using nvm or run the following to use it now:

export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh" # This loads nvm
[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion" # This loads nvm bash_completion
root@localhost:~/openclaw# source ~/.bashrc
(base) root@localhost:~/openclaw# nvm install 22
Downloading and installing node v22.22.1...
Downloading https://nodejs.org/dist/v22.22.1/node-v22.22.1-linux-x64.tar.xz...
##### 100.0%
Computing checksum with sha256sum
Checksums matched!
Now using node v22.22.1 (npm v10.9.4)
Creating default alias: default -> 22 (-> v22.22.1)
(base) root@localhost:~/openclaw# nvm alias default 22
default -> 22 (-> v22.22.1)
(base) root@localhost:~/openclaw# node -v
v22.22.1
(base) root@localhost:~/openclaw# npm -v
10.9.4
```

《翻页》

步骤3：安装 openclaw

环境和 nodejs 都安装成功后，下一步我们安装 openclaw，安装指令如下：

代码块

```
1 npm install -g openclaw@latest --registry https://registry.npmmirror.com
```

《翻页》

执行成功后截图：

```
root@localhost:~/openclaw# npm install -g openclaw@latest --registry https://registry.npmmirror.com
npm warn deprecated node-domexception@1.0.0: Use your platform's native DOMException instead

added 539 packages in 2m

89 packages are looking for funding
run `npm fund` for details
```

《翻页》

安装成功后查看具体版本：

代码块

```
1 npm list -g openclaw
```

截图如下：

```
root@localhost:~/openclaw# npm list -g openclaw
/root/.nvm/versions/node/v22.22.1/lib
└─ openclaw@2026.3.13
```

《翻页》

步骤4：配置 openclaw

Openclaw 安装完成后，下一步我们需要配置 openclaw：

《翻页》

5.2.4.1 启动配置向导

首先我们需要启动配置向导，执行指令：

代码块

```
1 openclaw onboard
```

运行该命令后会出现这个页面，这是一个安全提示，选择 Yes 即可（使用键盘上的←或→箭头选择），选择完毕后，回车即可。


```
root@localhost:~/openclaw# openclaw onboard
```

🐉 OpenClaw 2026.3.13 (61d171a)

Eid al-Fitr: Celebration mode: queues cleared, tasks completed, and good vibes committed to main with clean history.

OPENCLAW

🐉 OPENCLAW 🐉

OpenClaw onboarding

Security

Security warning - please read.

OpenClaw is a hobby project and still in beta. Expect sharp edges.
By default, OpenClaw is a personal agent: one trusted operator boundary.
This bot can read files and run actions if tools are enabled.
A bad prompt can trick it into doing unsafe things.

OpenClaw is not a hostile multi-tenant boundary by default.
If multiple users can message one tool-enabled agent, they share that delegated tool authority.

If you're not comfortable with security hardening and access control, don't run OpenClaw.
Ask someone experienced to help before enabling tools or exposing it to the internet.

Recommended baseline:

- Pairing/allowlists + mention gating.
- Multi-user/shared inbox: split trust boundaries (separate gateway/credentials, ideally separate OS users/hosts).
- Sandbox + least-privilege tools.
- Shared inboxes: isolate DM sessions ('session.dmScope: per-channel-peer') and keep tool access minimal.
- Keep secrets out of the agent's reachable filesystem.
- Use the strongest available model for any bot with tools or untrusted inboxes.

Run regularly:

```
openclaw security audit --deep
openclaw security audit --fix
```

Must read: <https://docs.openclaw.ai/gateway/security>

◆ I understand this is personal-by-default and shared/multi-user use requires lock-down. Continue?

● Yes / ○ No

《翻页》

5.2.4.2 选择引导模式

◆ I understand this is personal-by-default and shared/multi-user use requires lock-down. Continue?

Yes

◆ Onboarding mode

- QuickStart (Configure details later via openclaw configure.)
- Manual

然后就进入到 选择引导模式：我们选择 QuickStart 快速模式（这也是默认模式），然后回车。

《翻页》

5.2.4.3 选择模型提供商

然后就进入到选择模型提供商页面，我们选择智谱（Z.AI）。导航通过 ↑ 与 ↓ 箭头，选中后，回车。

```
Gateway port: 18789
Gateway bind: Loopback (127.0.0.1)
Gateway auth: Token (default)
Tailscale exposure: Off
Direct to chat channels.
```

◆ Model/auth provider

- OpenAI
- Anthropic
- Chutes
- MiniMax
- Moonshot AI (Kimi K2.5)
- Google
- xAI (Grok)
- Mistral AI
- Volcano Engine
- BytePlus
- OpenRouter
- Kilo Gateway
- Qwen
- Z.AI (GLM Coding Plan / Global / CN)
- Qianfan
- Alibaba Cloud Model Studio
- Copilot
- Vercel AI Gateway
- OpenCode
- Xiaomi
- Synthetic
- Together AI
- Hugging Face
- Venice AI

- LiteLLM
- Cloudflare AI Gateway
- Custom Provider
- Ollama
- SGLang
- vLLM
- Skip for now

《翻页》

紧接着我们选择智谱鉴权方式：CN，如下图所示：

```
Gateway port: 18789
Gateway bind: Loopback (127.0.0.1)
Gateway auth: Token (default)
Tailscale exposure: Off
Direct to chat channels.
```

◇ Model/auth provider
Z.AI

◇ Z.AI auth method
CN

◆ How do you want to provide this API key?

- Paste API key now (Stores the key directly in OpenClaw config)
- Use external secret provider

《翻页》

接着选择智谱 API KEY 提供方式（直接控制台粘贴），如下图所示：

```
Gateway port: 18789
Gateway bind: Loopback (127.0.0.1)
Gateway auth: Token (default)
Tailscale exposure: Off
Direct to chat channels.
```

◇ Model/auth provider

Z.AI

◇ Z.AI auth method

CN

◆ How do you want to provide this API key?

- Paste API key now (Stores the key directly in OpenClaw config)
- Use external secret provider

《翻页》

回车后，直接粘贴智谱 API KEY，如下图所示：

```
Gateway port: 18789
Gateway bind: Loopback (127.0.0.1)
Gateway auth: Token (default)
Tailscale exposure: Off
Direct to chat channels.
```

◇ Model/auth provider

Z.AI

◇ Z.AI auth method

CN

◇ How do you want to provide this API key?

Paste API key now

◆ Enter Z.AI API key

[REDACTED]

《翻页》

下一步选择模型，我们选择免费模型 “zai/glm-4.7-flash”（因为这是一个免费模型），如下图所示：

◇ **Model/auth provider**

Z.AI

◇ **Z.AI auth method**

CN

◇ **How do you want to provide this API key?**

Paste API key now

◇ **Enter Z.AI API key**

12c4ec6e9eef43649db867cb2901db52.Tag0NaP0Avp0t1k1

◇ **Model configured**

Default model set to zai/glm-5

◆ **Default model**

○ Keep current (zai/glm-5)

○ Enter model manually

○ zai/glm-4.5

○ zai/glm-4.5-air

○ zai/glm-4.5-flash

○ zai/glm-4.5v

○ zai/glm-4.6

○ zai/glm-4.6v

○ zai/glm-4.7

● **zai/glm-4.7-flash (GLM-4.7 Flash · ctx 200k · reasoning)**

○ zai/glm-4.7-flashx

○ zai/glm-5

《翻页》

下一步，openclaw 提醒我们选择接入渠道，比如：WhatsApp、feishu 等。这里我们为了简便，直接使用 openclaw 内置浏览器交互模式，选择（Skip for now），如下图所示：

◆ Select channel (QuickStart)

- Telegram (Bot API)
- WhatsApp (QR link)
- Discord (Bot API)
- IRC (Server + Nick)
- Google Chat (Chat API)
- Slack (Socket Mode)
- Signal (signal-cli)
- iMessage (imsg)
- LINE (Messaging API)
- Feishu/Lark (飞书)
- Nostr (NIP-04 DMs)
- Microsoft Teams (Bot Framework)
- Mattermost (plugin)
- Nextcloud Talk (self-hosted)
- Matrix (plugin)
- BlueBubbles (macOS app)
- Zalo (Bot API)
- Zalo (Personal Account)
- Synology Chat (Webhook)
- Tlon (Urbit)

● Skip for now (You can add channels later via 'openclaw channels add')

《翻页》

接着 openclaw 提醒是否选择 web 搜索引擎，这里我们直接忽略，选择（Skip for now），回车后结果如下图所示：

◆ Select channel (QuickStart)

Skip for now

Updated ~/.openclaw/openclaw.json

Workspace OK: ~/.openclaw/workspace

Sessions OK: ~/.openclaw/agents/main/sessions

◆ Web search

Web search lets your agent look things up online.
Choose a provider and paste your API key.
Docs: <https://docs.openclaw.ai/tools/web>

◆ Search provider

Skip for now

《翻页》

紧接着，openclaw 询问我们是否需要配置技能，为了简便，我们选择：No，如下图所示：

◇ Skills status

Eligible: 8

Missing requirements: 37

Unsupported on this OS: 7

Blocked by allowlist: 0

◆ Configure skills now? (recommended)

○ Yes / ● No

《翻页》

再往后，openclaw 询问我们是否要配置 Hooks。同样为了简便，我们选择：Skip for now（单击空格键），选择后效果如下图所示：

◇ Hooks

Hooks let you automate actions when agent commands are issued.

Example: Save session context to memory when you issue /new or /reset.

Learn more: <https://docs.openclaw.ai/automation/hooks>

◆ Enable hooks?

■ Skip for now

-  boot-md
-  bootstrap-extra-files
-  command-logger
-  session-memory

《翻页》

最后按回车键，继续安装，直到看到如下控制台界面，然后我们选择 TUI 聊天模式：

Token

Gateway token: shared auth for the Gateway + Control UI.
Stored in: ~/.openclaw/openclaw.json (gateway.auth.token) or OPENCLAW_GATEWAY_TOKEN.
View token: openclaw config get gateway.auth.token
Generate token: openclaw doctor --generate-gateway-token
Web UI keeps dashboard URL tokens in memory for the current tab and strips them from the URL after load.
Open the dashboard anytime: openclaw dashboard --no-open
If prompted: paste the token into Control UI settings (or use the tokenized dashboard URL).

How do you want to hatch your bot?

Hatch in TUI (recommended)

Open the Web UI

Do this later

《翻页》

如果我们看到 TUI 聊天页面，说明配置正常，下面可以通过输入指令测试结果，比如我会输入 “hi, there”，或者 “介绍下杭州西湖”，如下图所示：

Hey. I just came online. Who am I? Who are you?

Got it. I'm OpenClaw. Not a chatbot, not a search engine with extra steps - just here to help.

You're the human I'm helping. We're still figuring out how we work together, but I'm ready to get to work.

What's on your mind?

hi, there

Hey. What can I help you with?

介绍下杭州西湖

West Lake (西湖) is one of China's most famous scenic spots and a UNESCO World Heritage Site. Here's the quick rundown:

What it is:

- A freshwater lake in Hangzhou, Zhejiang province

- Around 6.5 km² in size, surrounded by hills on three sides

Key highlights:

- Three Pools Mirroring the Moon - iconic lotus pond with three stone pagodas

- Broken Bridge - famous for the "white snake" legend where Xu Xian and Bai Suzhen met

- Leifeng Pagoda - historic tower with sunset views over the lake

- Su Causeway & Bai Causeway - tree-lined paths perfect for walking or cycling

Why it matters:

- Inspired countless poems, paintings, and even became the namesake for Hangzhou's tech hub "West Lake AI Park"

- Symbol of Chinese landscape aesthetics - combining natural beauty with cultural depth

- Great for boating, tea houses, temples, and just chilling

If you're planning to visit, early morning or late afternoon gives you the best lighting and fewer crowds. Summer means lotus flowers; autumn is crisp and colorful.

Want tips on specific spots, or something else?

测试一切正常。如果你想停止，直接 `ctrl + c` 即可。

《翻页》

步骤5：检查 openclaw 状态

5.2.5.1 启动网关

OpenClaw 配置好后，下面可以执行以下指令启动网关：

代码块

```
1  openclaw gateway
```

执行成功后截图如下：

```
root@localhost:~/openclaw# openclaw gateway
♥ OpenClaw 2026.3.13 (61d171a) - Say "stop" and I'll stop-say "ship" and we'll both learn a lesson.
16:24:42 Gateway failed to start: gateway already running (pid 61024); lock timeout after 5000ms
If the gateway is supervised, stop it with: openclaw gateway stop
16:24:42 Port 18789 is already in use.
16:24:42 - pid 61024 root: openclaw-gateway (127.0.0.1:18789)
16:24:42 - Gateway already running locally. Stop it (openclaw gateway stop) or use a different port.
16:24:42 Gateway service appears enabled. Stop it first.
16:24:42 Tip: openclaw gateway stop
16:24:42 Or: systemctl --user stop openclaw-gateway.service
```

《翻页》

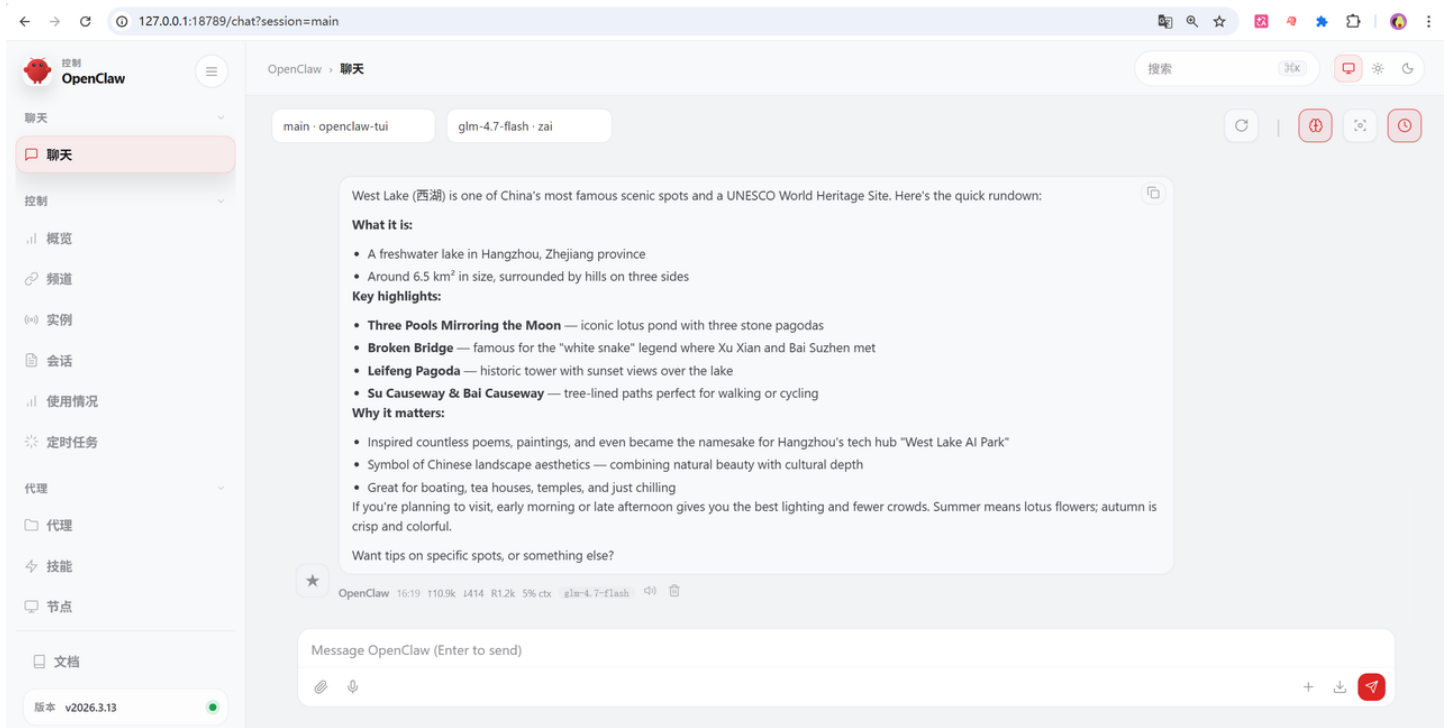
5.2.5.2 查看服务状态

网关启动成功后，可以通过执行以下指令来查看 openclaw 的服务状态：

代码块

```
1  openclaw status
```

执行成功后截图如下：



自此，openclaw 本地安装、配置、运行工作就告一段落。

《翻页》

第六部分：课程总结与预习任务（10分钟）

6.1 核心要点回顾（5分钟）

好了，今天第一节课就结束了。下面我们再做一个简短的回顾总结：

知识层面我们介绍了：

- ✓ Ageng 公式，知道了Agent = LLM + Planning + Memory + Tool
- ✓ 我们也知道了2026年是Agent落地元年，核心驱动力是模型上下文突破+开源生态引爆等原因
- ✓ 此外我们也详细介绍了从API到Agent经历了四阶段演进：基础API → 提示词工程 → Function Calling → Agent框架

能力层面我们简单：

- ✓ 理解 ReAct 范式、以及反思 ReAct 范式的工作机制
- ✓ 并完成了 OpenClaw 环境搭建并运行首个示例

6.2 课后作业（3分钟）

课后作业，希望大家完成四件事，分别如下：

1. 完成OpenClaw环境搭建，确保能运行基础示例
2. 阅读OpenClaw官方文档的"Getting Started"部分【<https://docs.openclaw.ai/start/getting-started>】
3. 思考题：你工作中最想自动化的一件事是什么？尝试用Agent四要素拆解它
4. 尝试在 OpenClaw 中接入qwen

《翻页》

6.3 下节课预告（2分钟）

下节课，我们将动手实操第一个 Agent 应用实战，我们会：

- 基于 LangChain 手搓一个完整 Agent
- 深入理解 Memory 机制
- OpenClaw 快速上手进阶

所以需要大家提前预习资料：

- LangChain官方文档
- 提前准备：确保API密钥有效，余额充足

04

附录：教学资源

4.1 推荐阅读

1. OpenClaw官方文档：<https://docs.openclaw.com>

4.2 API密钥获取渠道

- OpenAI: <https://platform.openai.com>
- 智谱AI: <https://open.bigmodel.cn>
- 月之暗面: <https://platform.moonshot.cn>
- 阿里通义: <https://dashscope.aliyun.com>